The Tradelatest Book

A Canonical Knowledge Book - reconstructing the repository as one sequence of ideas

Generated 2026-08-07 from docs/book/ (26 chapters, Parts I-VIII + Appendix)
This PDF is a reading export. The tracked, evolving source is docs/book/ in the repository - on any conflict, the repository wins.

Table of Contents

Front Matter

The Tradelatest Book

> **What this is.** A cover-to-cover reconstruction of this repository as a sequence of ideas, not a pile of files. Read it in order and you should understand *why* Tradelatest exists, *how* a candle becomes (or doesn't become) an order, and *where* every governance, research, and agent layer sits around that spine - without jumping between 900 documents to assemble the picture yourself.

> **What this is not.** A replacement for the code, for `CLAUDE.md`, or for the existing reference docs (`docs/reference/`), memory docs (`docs/memory/`), or topic docs (`docs/topics/`). This book is a *map*. Every chapter ends with the file paths that are the real, authoritative source - when the book and the code disagree, **the code wins** (per `CLAUDE.md` Section 0's hierarchy, which this book inherits). See Chapter 3 for exactly how the book relates to those other record systems.

> **Living document.** New chapters are added and existing ones are deepened over time; chapter numbers never change once assigned, so links stay stable. See "How this book evolves" below.

How to read this

- **New to the repo?** Start at Chapter 1 and read straight through Part V (chapters 01-15). That is the complete, coherent story of one candle's journey from raw OHLCV to an approved (or rejected) order - the spine everything else in the repo hangs off.
- **Already know the spine, need one layer?** Jump straight to the Part that covers it (table below) - each chapter is self-contained given the "What you need to already know" prerequisites it lists.
- **Looking for a specific concept** (CRT, Fusion, Ultron, Promotion, ...)? Use the Concept Index at the bottom of this file - it points to the *one* chapter that owns that concept's explanation, per the book's Canonical Concept Rule.

Table of Contents

Part I - Foundations

Why the system exists, what must never break, and how to navigate everything else.

04	Architecture at a Glance	Written

Part II - Market Understanding

Turning raw candles into a trustworthy, meaningful numeric surface.

#	Chapter	Status
05	Data Ingestion and the No-Lookahead Discipline	Written
06	The Market Ontology - Canonical Semantic Authority	Written
07	The Feature Pipeline and the Canonical Vector	Written

Part III - Market Semantics

Giving the numeric surface structure: states, transitions, and patterns.

#	Chapter	Status
08	The CRT State Machine - the Spine	Written
09	Interpreters and the Pattern Contract	Written

Part IV - Decision Making

Turning structure into a single execute/reject decision.

#	Chapter	Status
10	The Four Scoring Engines	Written
11	Fusion - Combining Independent Signals	Written
12	The Decision Engine - Semantic Approval Only	Written

Part V - Execution

Turning a decision into order geometry, and geometry into a capital-safe order.

#	Chapter	Status
13	The Execution Planner - From Decision to Order Geometry	Written
14	Ultron Risk Gate - the Final Capital Check	Written
15	Live Execution and INOUT	Written

Part VI - Governance

How a config earns the right to run in production, and how the repo keeps its own truth honest.

#	Chapter	Status
16	Config-First Doctrine and the Promotion Path	Written
17	Truth Maintenance - Findings, Closure, and Authority	Written
18	A Field Guide to docs/governance/	Orientation-level

Part VII - Research

How the repo tries - and mostly fails, on purpose and by design - to find a tradeable edge.

#	Chapter	Status
19	The Research Programs - Falsification as a Discipline	Orientation-level
20	The Research Platform - src/research/	Orientation-level

Part VIII - Agent Intelligence

The layers that let an LLM (or a human) operate the system through natural language.

#	Chapter	Status

21	The AI Automation Agent	Orientation-level
22	The Control Plane	Orientation-level
23	Multi-LLM Coordination	Orientation-level

Appendix

#	Chapter	Status
A1	Testing the System	Written
A2	Unresolved Questions (rollup)	Written

Concept Index (Canonical Concept Rule)

Each concept below has exactly one chapter that owns its explanation. Other chapters that touch it link back here instead of re-explaining it.

Concept	Canonical chapter
Market Ontology	Ch.06
Feature Pipeline / `CANONICAL_FEATURES`	Ch.07
CRT (state machine)	Ch.08
Gaussian / Zone Gate / RR engines	Ch.10
Fusion	Ch.11
Decision Engine	Ch.12
Execution Planner (`ExecutionPlannerV1_2`)	Ch.13
Ultron Risk Gate	Ch.14
Promotion / `PromotionManager`	Ch.16
Config Validation / `ConfigValidator`	Ch.16
Findings / Closure / Authority Ladder	Ch.17

Control Plane / `CommandSpec`	Ch.22
AI Automation Agent / `PLAN_REGISTRY`	Ch.21
Multi-LLM Protocol	Ch.23

How this book evolves

Per the book's own charter: when the repository changes, **only the affected chapter is updated** - chapter numbers and links are permanent. If a new concept needs a canonical home, it gets a new chapter number appended at the end of its Part's range (never a renumber). Governance/Research/ Agent Intelligence chapters (16-23) are marked "Orientation-level" - they are accurate but deliberately not exhaustive over their much larger source corpora (~250 files in `docs/governance/`, ~475 files across `src/research/` + `docs/research-readiness/`); deepening them is future book work, tracked in A2.

Source of truth: this book is generated understanding, not authority. On any conflict between a chapter and the code/config it describes, the code/config wins - see `CLAUDE.md` Section 0 and Section 4.0 for the repo's own precedence rules, which this book does not override.

Part I - Foundations

Chapter 01 - Why Tradelatest Exists

Part I - Foundations Status of this chapter: Written

Why this chapter exists

Before any code, config, or governance rule makes sense, one question needs an answer: what is this system actually *for*? Get that wrong and every later chapter reads as arbitrary engineering detail. Get it right and the rest of the book - four independent scoring engines, a fusion gate, a risk gate, a governance apparatus, a research program that mostly produces null results on purpose - snaps into place as one coherent design.

What problem it solves

Tradelatest reads M15 (15-minute) OHLCV candles for a handful of instruments, and for each candle asks: is this a valid, risk-bounded trading opportunity? If yes, it plans the trade's entry, stop, and target, and passes it through a final capital-safety check before anything is (in principle) executed. That is the entire functional surface of the system, stated in one sentence - but *how* it answers that question, and what it refuses to compromise on while answering it, is the real subject of this book.

What you need to already know

Nothing yet - this is the first chapter.

The idea

The one-paragraph goal

Tradelatest scores every M15 candle through four independent engines (CRT, Gaussian, Zone Gate, RR), fuses their scores under a weighted-completeness gate, plans entry/SL/TP via an execution planner, and approves the final position through a risk gate. Governance sits on top of all of this: no configuration reaches production without passing a validator and leaving an append-only audit trail. Everything is file-backed - there is no database, no message broker, no cloud dependency. This same one-paragraph summary appears in `CLAUDE.md` Section 1 because it is the fixed point the rest of the repository orbits.

Profit is not the primary objective - read that literally

The single most important, least obvious fact about this repository: **profit is explicitly not the top-priority goal.** The declared priority order, from `docs/architecture/goal.md`, is:

1. **Replay correctness** - the same inputs must always produce the same outputs.
2. **Explainability** - a human (or an LLM) must be able to say *why* a decision happened.
3. **Telemetry continuity** - historical data must remain interpretable as the system evolves.

4. **Advisory-AI** - language models assist and explain; they never trigger trades unsupervised.
5. Only then: economic performance, and even that comes with a caveat - *structure validity is not the same thing as execution validity.* A candle can be a textbook-valid CRT structure and still be a bad trade once realistic costs and slippage are applied.

This ordering explains a huge amount of what looks unusual elsewhere in the codebase: a research program (Part VII) that closes hypothesis after hypothesis as *null* and treats that as a successful outcome; a governance layer (Part VI) that will block a config from production over an un-auditable change even if backtests look good; and a decision spine that fails closed rather than guessing whenever something is ambiguous. The system is optimized to be **trustworthy first**, profitable second - because a system you cannot trust cannot be safely made profitable, but a system that lies convincingly about being profitable is actively dangerous.

The economic target, when it does apply

There is still a real, machine-readable economic objective - it's just deliberately subordinate and currently *advisory only*. It lives as the "Goal Layer" (id `G001`) inside the production config and states concrete targets: 20-40 trades/month (max 80), average reward:risk >= 2.0, win rate >= 0.35, max drawdown <= 10%, risk per trade 0.5%, expectancy >= 0.20R, M15 execution informed by H1 structure, reaction-only entries with human execution in the loop. Every backtest computes a `goal_report` against these targets, but nothing currently *enforces* them - there is a `goal.enforce` flag in the schema, and it stays off by design, because the system's honestly-measured current performance (see the Research Programs in Part VII) does not yet clear these targets. Turning that flag on prematurely would be exactly the kind of dishonesty the priority order above exists to prevent.

Pattern interpreters are measured, never asserted

One more design choice worth internalizing early: the system supports "interpreters" - modules that claim to recognize classical chart patterns (Point & Figure, Wyckoff, Market Profile, Order Flow). None of these are treated as intelligent by assumption. Each must satisfy an explicit Interpreter Contract and is *measured* against the Goal Layer via a forward-walk + qualification gate before it earns any influence - see Chapter 9. This is the same epistemic discipline as the priority order above, applied at the level of an individual idea rather than the whole system.

Classification

Concept	Status
The one-paragraph goal (4 engines -> fusion -> decision -> execution -> risk gate)	Production
Goal Layer / `G001` economic targets	Production artifact, **advisory-only** (measure, don't gate)
`goal.enforce` hard-gating	Built, **dormant by design**

Interpreter Contract-qualified pattern interpreters	Research

Authoritative sources

- `docs/architecture/goal.md` Section 1, Section 1.1 - the north-star statement and the Goal Layer targets, in full.
- `src/config_layer/goal_schema.py`, `src/config_layer/goal_validator.py` - the Goal Layer's code.
- `docs/topics/goal-layer.md` - the concept-level topic doc (code <-> tests <-> entry points).
- `CLAUDE.md` Section 1 - the same one-paragraph summary, kept in sync by convention.

Unresolved questions

None for this chapter - `goal.md` is a well-evidenced, internally consistent primary source.

Previous: - (first chapter) | **Next:** Chapter 02 - The Invariants and the Happy Flow **Related:** Chapter 19 - The Research Programs (what "measured, never asserted" looks like in practice) | Chapter 9 - Interpreters and the Pattern Contract **Memory:** none - this chapter summarizes a single primary doc rather than reverse-engineered code.

Chapter 02 - The Invariants and the Happy Flow

Part I - Foundations Status of this chapter: Written

Why this chapter exists

Chapter 1 established *why* the system exists and what it refuses to compromise on in principle. This chapter makes that concrete: the exact shape of a candle's journey through the system (the "happy flow"), and the seven hard rules ("invariants") that every later chapter's design decisions trace back to. Every subsequent Part of this book is, in effect, a zoomed-in view of one segment of the diagram below.

What problem it solves

Without a fixed reference diagram, it's easy to lose track of what's on the synchronous decision path versus what merely *feeds* it asynchronously. This chapter draws that line once, clearly, so later chapters don't have to re-justify it.

What you need to already know

Chapter 1 - the priority order (replay correctness first, profit last) is the reason these particular seven invariants were chosen.

The idea

The happy flow

One M15 candle closes. From there:

```
Candle -> [4 engines score independently] -> Fusion (all-four-or-reject) -> Decision
(threshold -> ACCEPT/REJECT) -> Execution Planner (entry/SL/TP/RR/TTL) -> Risk Gate
(Ultron) -> order out, or a logged rejection at any stage
```

In code, this is the **decision spine**:

```
EngineRunner -> FusionEngine.evaluate -> DecisionEngine -> ExecutionPlannerV1_2 ->
UltronRiskGate.evaluate
```

Three of the four engines score the candle as a single number each. The fourth, CRT, is different in kind: internally it's not a number generator, it's a **state machine** with nine legal states, walking a lifecycle:

```
RANGE -> SWEEP -> DISPLACEMENT -> EXPANSION -> RETEST -> EXECUTION -> RESOLUTION
```

plus two branches: `RANGE <-> SHADOW_PENDING -> SWEEP` (an early-detection branch) and `EXPANSION -> EXPIRED -> RANGE` (a TTL-based soft-archive branch when a setup goes stale before retesting). CRT's *score* is what feeds Fusion; CRT's *state* is what makes it "the spine" - Part III is dedicated to it.

The seven invariants

These are the rules that must never break, regardless of how much throughput or profit a change promises (`docs/architecture/goal.md` Section 3):

1. **Deterministic replay.** Same inputs -> same outputs, always. This is invariant #1 for a reason: nothing else in the book - governance, research, agent advice - is trustworthy if replay isn't.
2. **All four engines or nothing.** `EXPECTED_ENGINES = {"crt", "gaussian", "zone_gate", "rr"}` in `EngineRunner`. If any engine is missing, the candle is rejected outright - there is no silent partial fusion.
3. **The LLM is advice, never a trigger.** Any LLM-backed gate returns a neutral score after repeated failures rather than blocking or hallucinating a decision. A code path that cannot tolerate that neutral fallback is considered broken by design.
4. **No config reaches production without governance.** Every promotion needs an approved `ValidationReport`, a SHA-256 config hash, and an append-only `promotion_log.jsonl` entry - see Chapter 16.
5. **Telemetry is additive only.** Fields get renamed forward, never silently removed - old JSONL records must stay interpretable.
6. **CRT state moves only along legal transitions.** The nine-state graph above is the *entire* legal transition set - there is no free-form state graph.
7. **No database, no broker, no cloud dependency.** Everything is file-backed. A request to "add a table" or "use the ORM" is a convention break, not a valid design option.

Every change is additionally scored against five governance questions: is it still deterministic? Still comparable across runs? Auditable later? Can an LLM reason about it? Is execution authority still isolated? A change that fails any of these needs explicit sign-off, not silent acceptance.

The deviation policy

Not every deviation from "the ideal" is forbidden - the system distinguishes three tiers: - [OK] **Acceptable** - improves throughput, speed, or quality while keeping all seven invariants intact. - [!] **Needs a flag** - trades off an invariant for throughput (must be explicit, reversible, and visible). - [NO] **Not acceptable** - breaks determinism, governance, the LLM-as-advice rule, telemetry continuity, or partial-fusion protection, *regardless* of how much throughput or speed it buys.

The async "kitchen feeders"

Four subsystems sit beside the synchronous spine, feeding it without ever calling into it directly - they communicate only through files, never direct function calls:

- **Governance** - `AutoTuner -> ConfigValidator -> PromotionManager` writes new production configs that the spine reads at its next load. See Part VI.

- **Training** - raw trade JSONL -> `DatasetValidator -> Trainer -> ModelRegistry` writes model artifacts that engines (mainly Gaussian and BitNet's zone gate) read. See Chapter 10.
- **AI Agent** - either drives the spine through a backtest loop (Pipeline mode) or taps it read-only at the Fusion/Decision step (Copilot mode) - it can never mutate a decision in flight. See Chapter 21.
- **INOUT** - a parallel live-execution strategy rail that joins the spine only at the very last step, `UltronRiskGate.evaluate()` - and is currently archived (`archive/inout_legacy/`). See Chapter 15.

The full seven-step walk (data ingest -> feature pipeline -> scoring engines -> fusion & decision -> execution planner -> risk gate -> execution), with per-step module/entry-point/failure-mode detail and a Mermaid swim-lane diagram, lives in `docs/architecture/signal-flow.md` - this chapter's diagram is the compressed version; that file is the canonical one.

Classification

Concept	Status
The 7-step synchronous spine	Production
The 9-state CRT lifecycle	Production, **CLOSED** (see Ch.08)
The 7 invariants + 5 governance questions	Doctrine - enforced by review discipline, not (all) by code
Governance / Training / Agent / INOUT feeders	Production (Governance, Training, Agent) / **Archived** (INOUT)

Authoritative sources

- `docs/architecture/goal.md` Section 2-Section 4 - the happy flow, invariants, and deviation policy verbatim.
- `docs/architecture/signal-flow.md` - the authoritative 7-step walk with module/entry/failure-mode detail per step and the Mermaid swim-lane diagram (Section 4).
- `src/core/engine_runner.py` - `EXPECTED_ENGINES` and the completeness gate.
- `src/config_layer/crt_engine_v2.py`, `docs/architecture/event-taxonomy.md` Section 3 - the CRT transition graph, also re-exported at `state_topology.py:109`.

Unresolved questions

None - this chapter compresses two internally consistent, well-cross-referenced primary sources.

Previous: Chapter 01 - Why Tradelatest Exists | **Next:** Chapter 03 - How This Book Fits the Repository's Knowledge **Related:** Chapter 08 - The CRT State Machine | Chapter 16 - Config-First Doctrine and the

Promotion Path **Memory:** `docs/memory/architecture-memory.md` (deep companion, load only when a full cross-package map is needed).

Chapter 03 - How This Book Fits the Repository's Knowledge

Part I - Foundations Status of this chapter: Written

Why this chapter exists

This repository already has more documentation than most codebases have code: ~900 files under `docs/`, a 300+-file governance ledger, a memory hierarchy, a topics index, findings registries. A reader could reasonably ask "why does this need a *book* on top of all that?" This chapter answers that directly, and - because the book borrows vocabulary from those existing systems - defines the classification labels ("Production", "Experimental", "Research", ...) used in every chapter from here on, mapped honestly onto the repo's own status vocabulary rather than invented fresh.

What problem it solves

Without this chapter, a reader bounces between `CLAUDE.md`, `docs/knowledge-map.md`, `docs/topics/readme.md`, and `docs/memory/README.md` to figure out which of several overlapping "index" documents to trust for what. This chapter is that decision, made once.

What you need to already know

Chapter 1 and Chapter 2 - the substance this book organizes.

The idea

The repository already has record systems - this book is a new one, not a replacement

`docs/knowledge-map.md` names nine existing "record systems," each authoritative for a different question: the SESSION LOG (`assistant_project.md`, "what actually happened, dated"), Plans (`docs/plans/`, "what was designed"), Analysis (`docs/analysis/`, "point-in-time studies - evidence, not current truth"), Findings (`docs/current-findings.md`, "current validated conclusions"), Research families, Codebase structure maps, Topics, a Timeline (the date-join across the others), and Research substrates (frozen per-instrument datasets). None of these are *sequential* - each answers one kind of question well, but none walks a reader start-to-finish through the ideas.

This book is a tenth: the sequencing layer. It doesn't replace any of the above - it points into them, in order, the way a textbook points into a language's reference manual without reproducing it. Concretely: this book cites the SESSION LOG and findings for *what happened and what's proven*, `docs/topics/` for *the grounded per-concept picture* (code <-> tests <-> entry points), and `docs/memory/` for *the reading order into source* when you're about to edit a subsystem. If you're about to change code, `docs/memory/` is still the more direct next stop - see below.

The memory hierarchy still governs when you're about to edit code

`CLAUDE.md` Section 0 fixes a hierarchy: `CLAUDE.md` -> the matching `docs/memory/*-memory.md` companion -> deep companions (`docs/architecture/`, `docs/reference/`) -> **source code (always wins)**. This book sits *alongside* that hierarchy, not above it: reading a book chapter is how you learn what a subsystem is and why it exists; loading the matching memory doc is still the step you take immediately before editing that subsystem's code. The six memory docs and the `src/` trees they route to:

Memory doc	Routes to	Nearest book chapter
`agent-memory.md`	`src/agent/`	Ch.21
`runtime-memory.md`	`src/runtime/`	Ch.02, Ch.05
`feature-memory.md`	`src/features/`	Ch.07
`engine-memory.md`	`src/engines/`	Ch.10
`governance-memory.md`	`src/governance/`, `config_validator.py`	Ch.16
`architecture-memory.md`	multi-package, full spine	Ch.02, Ch.04

Topics vs. this book

`docs/topics/` (26 files) and this book overlap in subject but not in shape. A topic doc is a single, continuously-updated concept dossier - code anchor, entry points, tests, status, one file, kept in sync with code on *every* touching turn (`CLAUDE.md` Section 6.4). A book chapter is a fixed point in a narrative sequence - it teaches the concept in the order a new reader needs to encounter it, and links to the topic doc for the always-current, code-synced detail. Where both exist for the same concept (e.g. CRT, Fusion+Decision, Execution Planning, Ultron, the Agent), **the topic doc is the fresher, code-verified source**; this book's chapter is the narrative on-ramp to it.

The classification vocabulary this book uses

Per the book's own brief, every concept is classified as one of: **Production, Experimental, Research, Legacy, Deprecated, Partial, Unknown**. The repository doesn't use these exact seven words - it has its own, more granular status vocabulary, built up across different subsystems. This book maps onto that vocabulary rather than inventing a parallel one:

Book label	Repo-native equivalent(s) it draws on
Production	Wired into the live decision spine; `docs/topics/` status `living`; Closure & Authority Index `CLOSED` (fully proven boundary)

Experimental	`docs/governance/` closure status `AUDITED` (reviewed, not certified) or `COMPLETE` (a sub-step, not the whole domain); config-gated but not the active default
Research	Lives under `src/research/`, `docs/research-readiness/`; findings with confidence `Likely`/`Possible`; Authority Ladder tier "information exists" or "economic usefulness exists" but not "authority earned" (Section 6.5)
Legacy	`docs/topics/` status `DORMANT` - real code, off the live path, sidecar/inert/orphaned
Deprecated	Explicitly superseded per a finding's `SUPERSEDED`/`INVALIDATED_BY` marker (CLAUDE.md Section 6.2 rule 4 - the row is kept, never deleted)
Partial	Built but unwired (e.g. TradeNet v2, F-005) or `docs/topics/` status `stub`
Unknown	No finding, topic, or closure artifact covers it - flagged in each chapter's Unresolved Questions, never guessed

A chapter's "Classification" section always uses this table's left column, so the label means the same thing everywhere in the book even though the underlying evidence format differs by subsystem.

Classification

Not applicable - this chapter defines classification rather than applying it.

Authoritative sources

- `docs/knowledge-map.md` - the nine existing record systems and how they connect (read in full for this chapter).
- `docs/memory/README.md` - the memory hierarchy and task->doc routing table.
- `docs/topics/readme.md` - the topic index, its status vocabulary (`living`/`stub`/`DORMANT`), and its explicit relationship to the idea-governance "Bricks" framework (kept parallel, not merged, by a 2026-06-01 decision).
- `CLAUDE.md` Section 0 (Architecture Memory Policy), Section 2 (Companion Documentation table).
- `docs/governance/closure_authority_index.json` and its `CLOSED`/`AUDITED`/`COMPLETE`/`OPEN` vocabulary (see Chapter 17 for the full explanation of that index).

Unresolved questions

None - this is a navigational chapter over existing, verified index documents.

Previous: Chapter 02 - The Invariants and the Happy Flow | **Next:** Chapter 04 - Architecture at a Glance
Related: Chapter 17 - Truth Maintenance (the closure/authority vocabulary in full) **Memory:** `docs/memory/README.md` (this chapter's primary subject).

Chapter 04 - Architecture at a Glance

Part I - Foundations Status of this chapter: Written

Why this chapter exists

Chapters 1-3 gave the *why* and the *how to navigate*. Before descending into Part II's per-concept chapters, a reader needs one honest picture of the *shape* of the codebase: how big each piece is, what depends on what, and where the boundaries are. This chapter is that picture, built from the repo's own auto-generated architecture artifacts rather than re-derived by hand (so it stays verifiable against `docs/architecture/code-map.generated.md` and `module-roles.generated.md` whenever they're regenerated).

What problem it solves

`src/` alone has 1,400+ files across 36 subpackages. Without a map, "where does this change land?" is a search problem every time. This chapter turns it into a lookup.

What you need to already know

Chapter 2 - the decision spine this map is organized around.

The idea

The repository is not one clean package

At the repo root, alongside the real source tree, sit years of working-repo residue: PDFs, zip backups, ad-hoc probe scripts, planning artifacts. The load-bearing top-level entries are:

Directory	Purpose	Approx. size
`src/`	The application itself	1,400+ files, 36 subpackages
`scripts/`	CLI/operational entry points (thin wrappers over `src/`)	500+ files, 18 subdirs
`docs/`	The knowledge corpus this book sits inside	~900 files
`configs/`	Production/research/experimental config surface	~65 files

`tests/`	Pytest suite, mirrors `src/` structure	~1,200 files
`multi_llm/`	Multi-LLM coordination protocol (see Ch.23)	~33 files
`data/`, `models/`, `results/`	Market data, trained artifacts, run outputs (mostly generated)	large, non-canonical
`CLAUDE.md`	The root AI-agent operating manual	-
`assistant_project.md`	The append-only session log - the spine of *history* (see Ch.03)	-

The 36 `src/` subpackages

Grouped roughly by the Part of this book that covers them (a subpackage not yet covered by a written chapter is marked so honestly rather than guessed at):

Subpackage	Covers	Book chapter
`features/`	Feature schema/pipeline/formulas	Ch.07
`config_layer/`	CRT engine, config schema/validation/goal schema, execution planner	Ch.06, Ch.08, Ch.13
`engines/`	The four scoring engines	Ch.10
`interpreters/`	Pattern interpreter contract (P&F, Wyckoff, ...)	Ch.09
`core/`	`EngineRunner`, `FusionEngine`, `DecisionEngine`, `UltronRiskGate`, core protocols	Ch.11, Ch.12, Ch.14
`runtime/`	Backtest engine (`backtest_v2.py`), live engine hook	Ch.05
`inout/`, `live/`, `execution/`	Live-execution I/O (`inout` currently archived)	Ch.15
`governance/`	Promotion, validators, registries, audit trail	Ch.16

`agent/`	AI automation agent (modes, tools, plan compiler)	Ch.21
`control_plane/`	Command registry / orchestration	Ch.22
`multi_llm/`	Multi-LLM coordination runtime	Ch.23
`research/`	Research programs (416 files - the largest subpackage by far)	Ch.19, Ch.20
`bitnet/`	Low-bit zone-gate model	Ch.10
`training/`	Model training pipelines	Ch.16 (feeds the Training kitchen feeder)
`expansion/`, `journal/`, `replay/`, `portfolio/`, `regime/`, `retrieval/`, `strategies/`, `msip/`, `scanner/`, `feedback/`, `cognitive/`, `monitoring/`, `data_ingestion/`, `uat/`, `llm_research/`, `analytics/`, `events/`, `validation_access/`, `search/`, `logs/`, `ui/`, `utils/`	Sidecar, offline, or supporting subsystems - several are explicitly `DORMANT` per `docs/topics/readme.md` (off the live decision path by design: Cognitive bus, Strategies S01-S10, Scanner, Feedback, LLM research, Monitoring, Journal, Data ingestion, UAT)	Not yet individually chaptered - see A2

The dependency shape, at the top level

`docs/architecture/code-map.generated.md`'s L0 package-dependency graph (auto-regenerated by `scripts/analysis/gen_code_map.py`) records edges like: `agent -> config_layer`, `agent -> control_plane`, `agent -> core`, `agent -> governance`, `agent -> runtime` (the agent depends on almost everything it can act on, never the reverse); and `bitnet -> engines`, `bitnet -> features`, `bitnet -> utils` (BitNet is a downstream consumer of the engine/feature layer, not upstream of it - this matters when reading Chapter 10's account of the Zone Gate engine). The full 35-package graph is larger than is useful to reproduce here; treat this chapter's excerpt as an orientation and the generated file as the source of truth, since it's regenerated directly from imports rather than hand-maintained.

Two companion generated docs worth knowing about

- `docs/architecture/module-roles.generated.md` - a one-line role per `src/` module, sourced from each module's own docstring. Useful as a fast "what does this file do" lookup when a chapter's "Authoritative sources" list points you at an unfamiliar path.
- `docs/reference/architecture.md` - a more traditional onboarding doc (tech stack with versions, directory structure, data-flow diagrams for the backtest/live/governance/agent paths, design patterns, external integrations, `.env`/config overview, execution entry points). This book does not reproduce that doc's content - treat it as the reference companion to this chapter's narrative framing.

Classification

Concept	Status
`src/` 36-package structure	Production (the whole tree ships)
`research/` (416 files)	Mixed - see Ch.19/Ch.20
The 9 subpackages marked `DORMANT` above	Legacy
Generated architecture docs (`code-map`, `module-roles`)	Production tooling, regenerate-don't-hand-edit

Authoritative sources

- `docs/reference/architecture.md` - tech stack, directory tree, data-flow diagrams, design patterns, config overview, entry points (headers verified this session; full content not reproduced here by design - see Ch.03's anti-duplication policy).
- `docs/architecture/code-map.generated.md` - the authoritative import graph (regenerate via `python scripts/analysis/gen_code_map.py`).
- `docs/architecture/module-roles.generated.md` - per-module one-line roles.
- `docs/topics/readme.md` - the `DORMANT` subsystem table.

Unresolved questions

- The 22 subpackages in the "not yet individually chaptered" row above are real, verified to exist, but this pass didn't have room to give each its own chapter. Tracked in A2.
- `docs/reference/architecture.md`'s full body (design patterns, external integrations, entry points) was header-verified but not read in full this session - a future pass should confirm this chapter's summary against the full text.

Previous: Chapter 03 - How This Book Fits | **Next:** Chapter 05 - Data Ingestion and the No-Lookahead Discipline **Related:** Chapter 02 - The Invariants and the Happy Flow **Memory:** `docs/memory/architecture-memory.md` (deep companion for full cross-package detail).

Part II - Market Understanding

Chapter 05 - Data Ingestion and the No-Lookahead Discipline

Part II - Market Understanding Status of this chapter: Written

Why this chapter exists

Everything downstream - features, engines, fusion, decisions - is only as trustworthy as the candle stream feeding it. Invariant #1 from Chapter 2, deterministic replay, is only meaningful if the data itself never leaks future information into the past. This chapter covers Step 1 of the spine: how a candle enters the system, and what stops it from cheating.

What problem it solves

A backtest that can see the future is not a backtest - it's a curve-fit generator. This chapter explains the layered defense against that failure mode, and is honest about where that defense is strong versus merely adequate.

What you need to already know

Chapter 2's seven-step spine - this chapter is Step 1 of it.

The idea

Where a candle comes from

In backtest, `runtime/backtest_v2.py`'s `CandleLoader.stream()` reads historical OHLCV and yields candles one at a time, in order. In live mode, `inout/scanner.py` plays the equivalent role, reading from a broker/data feed. Either way, the unit that leaves this step and enters the feature pipeline is a single `Candle`.

The four-layer integrity stack

No-lookahead isn't one gate - it's the *conjunction* of four layers, and that framing matters: a gap in any one layer doesn't necessarily mean data leaked, but it does mean the safety margin is thinner than the others.

- **L1 - schema.** Every candle is validated against the expected OHLCV shape before it's used.
- **L2 - temporal.** Candles must arrive in strictly increasing chronological order; out-of-order or duplicate timestamps are rejected. This backstop is inline and always-on, everywhere `stream()` is consumed.
- **L3 - dataset-level pre-flight.** A heavier, whole-dataset integrity check (`dataset_integrity.validate_dataset`) that runs *before* a backtest starts, catching structural problems (gaps, session-boundary anomalies) that a candle-by-candle check can't see.

- **RT - real-time generator causal ordering.** In the live path, the generator itself is constructed to only ever emit a candle once it has actually closed.

The honest caveat: L3 doesn't run everywhere

A repo-wide census found that the L3 pre-flight check runs at only **two call sites**, both inside `backtest_v2` (`_preflight_dataset` and `validate_universe`). Every other consumer of `CandleLoader.stream()` - the entire `src/research/` pipeline, analytics, governance, replay, `config_validator`, and the broader `scripts/` research/training/analysis fleet - streams candles with only the always-on L1/L2 inline backstop, not L3. This was captured as a documentation correction: an earlier claim that L3 ran at "every backtest entry point" was itself an overclaim, now corrected. The scope of what this means is narrow and important to get right: **L1/L2 alone did still enforce schema and chronological ordering** on every one of those research runs - this is a *single-layer fragility* finding (L3's extra dataset-wide safety margin is concentrated in two call sites), not a claim that leakage occurred. It does not reopen or invalidate any research finding built on those paths.

Classification

Concept	Status
L1 (schema) / L2 (temporal) inline backstop	Production, always-on everywhere `stream()` is used
L3 dataset pre-flight	Production, but scope-limited to two `backtest_v2` call sites
RT causal ordering (live generator)	Production

Authoritative sources

- `docs/architecture/signal-flow.md` Section 1, Step 1 - the four-layer table with module/failure-mode detail.
- `src/runtime/backtest_v2.py` - `CandleLoader.stream()`, `_preflight_dataset`, `validate_universe`.
- `src/inout/scanner.py` - the live-mode equivalent.
- `docs/current-findings.md` - F-039 (the L3-scope correction, `Certain` confidence, docs/research-only authority).

Unresolved questions

None beyond what F-039 itself already scopes precisely - see the finding for exact call-site detail.

Previous: Chapter 04 - Architecture at a Glance | **Next:** Chapter 06 - The Market Ontology **Related:** Chapter 07 - The Feature Pipeline **Memory:** `docs/memory/runtime-memory.md`.

Chapter 06 - The Market Ontology: Canonical Semantic Authority

Part II - Market Understanding Status of this chapter: Written

Why this chapter exists

Before a candle can be scored, every quantity computed from it - a wick length, a displacement strength, a zone boundary - needs one, and only one, agreed-upon meaning across the whole codebase. This chapter covers the artifact that owns that meaning: `configs/formulas/market_ontology.yaml`. Understanding it first is what makes Chapter 7's feature vector, and every engine chapter after it, legible rather than a wall of unexplained formulas.

What problem it solves

Historically, this repository suffered from *definitional drift* - the same-sounding quantity (`wick_size`, `body_ratio`) computed two different ways in two different files, silently. The ontology exists to make that structurally impossible going forward: one canonical node per concept, with every consumer required to bind to it rather than re-derive it locally.

What you need to already know

Chapter 5 - the ontology's `base_inputs` are exactly the OHLCV fields that survive ingestion.

The idea

What the ontology actually is

`market_ontology.yaml` (version `1.4` as of this writing, ~2,600 lines) is the repository's declared "**permanent semantic authority**." It defines every market concept as a typed node: `primitives` (raw geometric building blocks), `feature_compositions`, `derived_metrics`, `rolling_indicators`, `temporal_context`, `structural_states`, `indicator_identities`, `execution_behaviours`, plus `invariants` and - importantly - `canonical_unknowns`, an explicit place to register a discovered-but-not-yet-understood market behavior rather than let it live only as a code comment or a TODO.

Every node carries a fixed field set: id, canonical name, aliases, a *knowledge_status* on a refinement ladder, its formula, its dependencies, and who produces/consumes it. The refinement ladder itself is worth internalizing, because it's the vocabulary this book borrows for describing how mature any given piece of market knowledge is:

```
UNKNOWN -> OBSERVED -> CHARACTERIZED -> MATHEMATICALLY_DEFINED -> FORMULA_DERIVED ->
VALIDATED -> PRODUCTION_CERTIFIED -> STABLE
```

A node only reaches `PRODUCTION_CERTIFIED` by demonstrating measured economic value - see the Authority Ladder in Chapter 17. Reaching a mathematically clean formula is necessary but never sufficient for that top tier.

Why "authority" here means something specific

The ontology's authority is "literal supersession" for *meaning* - when a semantic question arises, this file is where the answer originates, and code, config, research, and even historical findings are expected to synchronize to it, not the other way around. But that authority is bounded by two physical constraints, and both matter for anyone editing it:

1. **Frozen runtime keys.** The `primitives`, `feature_compositions`, and `derived_metrics` sections are read at *import time* by `crt_engine_v2.py` (via `fm_resolve.bind_phase2_crt_callables()`). They can only grow additively - removing or renaming a key there is a breaking change to running code, not a documentation edit.
2. **Behavior changes still need governance.** "Ontology first" governs where a semantic change *originates*, not an automatic license to mutate runtime behavior. Any change that actually alters production output still has to clear the parity-proof and promotion gate from Chapter 16 - the ontology can't bypass governance, it feeds it.

Downstream reach

Every consumer that computes market meaning is expected to bind to this file rather than re-derive it: `crt_engine_v2.py` at import time (frozen sections), the feature registry / formula registry facade that Chapter 7 depends on, and - per the ontology's own evolution contract - effectively everything else in the pipeline traces its semantic meaning back here. This is why it's Chapter 6, not buried in an appendix: it's upstream of every later Part's vocabulary.

Classification

Concept	Status
Ontology as the semantic authority for meaning	Production, actively governed (CLAUDE.md Section 6.6)
Frozen runtime keys (`primitives`/`feature_compositions`/`derived_metrics`)	Production, additive-only
`canonical_unknowns` section	Explicit, honest placeholder for open questions - not a gap in the doctrine, a designed feature of it

Authoritative sources

- `configs/formulas/market_ontology.yaml` - the ontology itself (2,600+ lines, version 1.4, 2026-07-24).

- `docs/governance/MARKET_ONTOLOGY_EVOLUTION_CONTRACT.md` - the full evolution contract this chapter summarizes.
- `CLAUDE.md` Section 6.6 - the Canonical Market Ontology Evolution Contract (the two mechanical constraints, the refinement ladder, the Automatic Semantic Discovery ritual).
- `src/features/registry/` (formula registry facade), `src/config_layer/crt_engine_v2.py` (import-time binding).
- `docs/current-findings.md` F-047 - the ontology's authoritative status, source-verified.

Unresolved questions

None for this pass - the ontology's own evolution contract is unusually explicit about its own scope and limits, which made this chapter straightforward to write faithfully.

Previous: Chapter 05 - Data Ingestion and the No-Lookahead Discipline | **Next:** Chapter 07 - The Feature Pipeline and the Canonical Vector **Related:** Chapter 17 - Truth Maintenance (the Authority Ladder that governs when a node earns production certification) **Memory:** `docs/memory/feature-memory.md`.

Chapter 07 - The Feature Pipeline and the Canonical Vector

Part II - Market Understanding Status of this chapter: Written

Why this chapter exists

The four scoring engines in Part IV don't see a candle - they see a fixed-length numeric vector. This chapter covers where that vector comes from, what's in it, and - because this is where the book caught a live piece of documentation drift while researching it - exactly how many dimensions it actually has today, as opposed to what the reference doc still says.

What problem it solves

Explains the single most-consumed data contract in the codebase: `CANONICAL_FEATURES`. Every engine, BitNet, and most of the research pipeline binds to this vector's shape.

What you need to already know

Chapter 6 - the feature math this pipeline runs is derived from the ontology's `primitives`/`feature_compositions`/`derived_metrics`, not invented locally.

The idea

What the pipeline does

`src/features/feature_pipeline.py`'s `FeaturePipeline.run()` takes an enriched candle stream and produces `(enriched_df, vectors)` - a dataframe plus the fixed-length numeric vectors that engines actually consume. The vector's *shape* - which fields, in which order, at what dimensionality - is defined once, in `src/features/feature_schema.py`, as `CANONICAL_FEATURES`. Nothing downstream is supposed to reconstruct that shape independently; `core/engine_runner.py` imports it directly, as do `engines/zone_gate_engine.py`, `engines/heuristic_gaussian_engine.py`, and `crt_engine_v2.py`.

A live doc-drift, caught and flagged (not silently fixed)

`docs/reference/schemas.md` Section 4 still documents a **38-dimension, schema v3.0** feature tuple - `CANONICAL_FEATURE_DIM = 38`, with a single `macd_hist` field and a `wick_size` field. The code that actually runs today, `src/features/feature_schema.py`, is at **39 dimensions, schema v4.0** (`CANONICAL_FEATURE_DIM: int = 39`): `macd_hist` was split into `macd_hist_raw` and `macd_hist_z` (indices 18-19), and `wick_size` was renamed to `candle_range` (index 28). A `SCHEMA_V3_ALIASES` mapping exists solely so old v3 records can still be *read*; it is not what new code produces. The pipeline module's own docstring calls this out explicitly, warning readers to quote the code constant rather than a number, because

the docstring itself had briefly carried a stale "38" through the v2->v3->v4 migrations.

This is a textbook `DOC_DRIFT` case per `CLAUDE.md` Section 6.2 (code wins, doc needs fixing) - the fix itself (F-062, 2026-07-31) already shipped on the *code* side; what appears to remain is that `docs/reference/schemas.md`'s table wasn't updated to match. **This book flags it and points at the authoritative constant; per this chapter's own scope, it does not silently edit `schemas.md` as a side effect of being written** - that's a separate, explicit doc-drift turn for whoever picks it up next (tracked in A2).

The rule this leaves you with: when you need the current dimensionality or field order, read `src/features/feature_schema.py`'s `CANONICAL_FEATURES` constant directly. Don't quote a number from a doc - quote the constant.

Who consumes the vector

`core/engine_runner.py` is the hub: it builds the 39-dim vector once per candle and hands it to all four engines uniformly. This is what makes "all four engines or nothing" (Chapter 2's invariant #2) mechanically enforceable - every engine is looking at the exact same input surface, so a missing engine is a missing *scorer*, never a missing *input*.

Classification

Concept	Status
`feature_pipeline.py` / `FeaturePipeline.run()`	Production
`CANONICAL_FEATURES` @ 39-dim, schema v4.0 (`feature_schema.py`)	Production, current
`docs/reference/schemas.md`'s 38-dim v3.0 table	**Stale documentation** - `DOC_DRIFT`, code already fixed (F-062), doc not yet reconciled
`SCHEMA_V3_ALIASES`	Production, read-compatibility only - not what new code emits

Authoritative sources

- `src/features/feature_pipeline.py` - the pipeline entry point (`FeaturePipeline.run()`).
- `src/features/feature_schema.py` - **the single source of truth** for `CANONICAL_FEATURES` and `CANONICAL_FEATURE_DIM`.
- `docs/reference/schemas.md` Section 4 - the reference doc, currently lagging the code (flagged above).
- `docs/current-findings.md` - F-062 (the code-side rename that created this v3/v4 gap).
- `docs/topics/feature-schema.md` - the always-synced topic doc for this concept.

Unresolved questions

- **Should `docs/reference/schemas.md` Section 4 be updated to 39-dim/v4.0 now?** This is a real, low-risk `DOC_DRIFT` fix (per `CLAUDE.md` Section 6.2's gate calibration, an unambiguous stale-fact fix like this can be auto-fixed without user approval) - it was deliberately left out of this book-writing pass to keep this task's scope to the book itself. Recorded in A2 as the single most actionable follow-up this book surfaced.

Previous: Chapter 06 - The Market Ontology | **Next:** Chapter 08 - The CRT State Machine **Related:** Chapter 10 - The Four Scoring Engines (the vector's consumers) **Memory:** `docs/memory/feature-memory.md`.

Part III - Market Semantics

Chapter 08 - The CRT State Machine: the Spine

Part III - Market Semantics Status of this chapter: Written

Why this chapter exists

CRT (Candle Range Theory) is described everywhere in this repository as "the spine" - not because it's the most powerful of the four scoring engines, but because unlike the other three, it isn't a single number-producing function. It's a stateful lifecycle that gives a *sequence* of candles structural meaning, and that structure is what the rest of Part IV's decision-making ultimately reasons about.

What problem it solves

A single candle's numeric features (Chapter 7) can't tell you "this is currently mid-setup, three candles into a retest of a liquidity sweep." Only a state machine that persists across candles can. CRT is that state machine.

What you need to already know

Chapter 6 and Chapter 7 - CRT's frozen ontology sections and its input vector are both prerequisites already covered.

The idea

Two files named "CRT engine" - know which one you're reading

There are two distinct modules worth telling apart precisely, because confusing them is an easy mistake:

- **`src/config_layer/crt_engine_v2.py`** - the full deterministic state machine. Internally composed as `RangeDetector -> StateMachine -> (internal risk/execution helpers) -> ...`, and this is the module that binds the ontology's frozen sections at import time (see Chapter 6).
- **`src/engines/crt_engine.py`** - a thinner wrapper that `EngineRunner` (Part IV) actually calls as one of the four scoring engines. Its `compute()` delegates to `engines/scoring_engine.py`'s `compute_scores`, and what it returns to Fusion is a single number - CRT's *score*, not its full state.

So: the **state** lives in `crt_engine_v2.py`; the **score** that Fusion sees comes from the thinner `engines/crt_engine.py` wrapper. Both matter, but they answer different questions - "where is this setup in its lifecycle" versus "how good does this setup look right now."

The nine-state lifecycle

Already introduced in Chapter 2:

```
RANGE -> SWEEP -> DISPLACEMENT -> EXPANSION -> RETEST -> EXECUTION -> RESOLUTION
```

plus two branches: - `RANGE <-> SHADOW_PENDING -> SWEEP` - an early-detection branch that lets the engine track a candidate setup before it's confirmed. - `EXPANSION -> EXPIRED -> RANGE` - a TTL-based soft-archive: a setup that goes stale before reaching RETEST ages out rather than lingering indefinitely.

This is the **entire** legal transition graph (invariant #6 from Chapter 2) - there is no path through the state machine that isn't one of these edges. The authoritative transition table lives at `state_identity.py:69` (extracted from `crt_engine_v2` to break an import cycle, re-exported for compatibility) with an immutable view at `state_topology.py:109`, and is narrated in `docs/architecture/event-taxonomy.md` Section 3.

CRT is a closed research surface - what that does and doesn't mean

CRT carries the repository's only **CLOSED** status in the Closure & Authority Index (see Chapter 17 for what that index means in general). The declared boundary is narrow and precise: *OHLCV -> CRT `TRADE_OPENED`, and only that.* It does not mean "CRT is proven profitable" - the Research Programs in Part VII repeatedly test CRT-adjacent hypotheses (completed sweep->displacement->retest sequences, weekly-calendar sweep variants) and find no economic edge. CLOSED here means: the *mechanism itself* - candle in, state transitions, `TRADE_OPENED` event out - has been audited end-to-end and is considered a stable, trustworthy piece of machinery. Whether what it detects is *worth trading* is a separate, open question, owned by Part VII.

Classification

Concept	Status
The 9-state CRT lifecycle + transition graph	Production, **CLOSED** (mechanism boundary only)
`crt_engine_v2.py` (full state machine)	Production
`engines/crt_engine.py` (score wrapper for `EngineRunner`)	Production
"Is a completed CRT structure economically tradeable"	Research - repeatedly tested, currently null (see Ch.19)

Authoritative sources

- `src/config_layer/crt_engine_v2.py` - the full state machine.
- `src/engines/crt_engine.py`, `src/engines/scoring_engine.py` - the `EngineRunner`-facing score wrapper.
- `src/config_layer/state_identity.py:69`, `state_topology.py:109` - the authoritative transition graph.
- `docs/architecture/event-taxonomy.md` Section 3 - the narrated transition table.
- `docs/governance/crt_closure_report.md` - the CLOSED boundary declaration and its exact scope.
- `docs/topics/crt-spine.md` - the always-synced topic doc.

Unresolved questions

None for the mechanism itself - CRT is the single most thoroughly audited surface in the repository. Its *economic* status is intentionally left open here and answered properly in Part VII.

Previous: Chapter 07 - The Feature Pipeline | **Next:** Chapter 09 - Interpreters and the Pattern Contract **Related:** Chapter 10 - The Four Scoring Engines | Chapter 17 - Truth Maintenance | Chapter 19 - The Research Programs **Memory:** `docs/memory/engine-memory.md`.

Chapter 09 - Interpreters and the Pattern Contract

Part III - Market Semantics Status of this chapter: Written

Why this chapter exists

CRT (Chapter 8) is one way of giving candle sequences structural meaning. It is not the only pattern vocabulary a trader might reach for - Point & Figure, Wyckoff, Market Profile, Order Flow are all classical alternatives. This chapter covers how this repository lets such ideas in *without* letting them claim intelligence they haven't earned.

What problem it solves

Prevents "we added a pattern detector" from silently becoming "we trust this pattern detector." Every interpreter has to prove itself the same way, before it can influence anything.

What you need to already know

Chapter 1's "measured, never asserted" principle, and Chapter 8 - interpreters are a parallel idea to CRT, not a replacement.

The idea

The Interpreter Contract (Level 4)

`src/interpreters/contract.py`, `adapter.py`, and `reference.py` define a fixed contract any pattern-recognition module must satisfy before it's allowed to matter. A candidate pattern is wrapped as an `InterpreterHypothesis` and run through the same `forward_walk` evaluation plus `QualificationGate` machinery used elsewhere in the research pipeline - the same yardstick, reused, not a bespoke one invented per interpreter. This is deliberate: it means "does this pattern help" is answered the same way every time, and a pattern can't get a friendlier evaluation just because its author believed in it more.

The one interpreter that's actually been run through it

Point & Figure (P&F, implementation id `PNF-v1`, double-top/double-bottom patterns) is the first interpreter to complete this process on real data. The result was a clean **REJECT**: shadow-measured on crypto majors, it carried no standalone edge, and scored worse than random controls (a negative delta-expectancy of about -0.039). This is reported here not as a failure of the interpreter framework but as evidence the framework *works as intended* - it correctly declined to grant authority to an idea that didn't earn it, exactly the discipline Chapter 1 described. Per the repository's own research discipline, this closes P&F under the current ontology; reopening it needs a genuinely new ontology, not another sweep of the same one.

What this means for other interpreters

Nothing else currently in `src/interpreters/` has cleared the contract - until one does, the honest status of the whole subsystem is: infrastructure exists, one candidate has been tested and rejected, the rest are unevaluated. None of them influence the live decision spine.

Classification

Concept	Status
Interpreter Contract (`contract.py`/`adapter.py`/`reference.py`)	Production infrastructure
P&F / `PNF-v1`	Research - **tested, REJECTED** (F-028)
Wyckoff, Market Profile, Order Flow interpreters	Unknown / not yet run through the contract

Authoritative sources

- `src/interpreters/contract.py`, `adapter.py`, `reference.py` - the contract itself.
- `docs/topics/interpreter-contract.md` - the always-synced topic doc, entry point, and test references.
- `docs/current-findings.md` F-028 - the P&F result, with its exact scope and confidence.
- `docs/architecture/goal.md` - the "measured against the Goal Layer, never asserted" framing this chapter draws its principle from.

Unresolved questions

- Whether Wyckoff/Market Profile/Order Flow interpreters have any code beyond scaffolding, and if so how far along the contract pipeline they are, wasn't verified in this pass - flagged in A2.

Previous: Chapter 08 - The CRT State Machine | **Next:** Chapter 10 - The Four Scoring Engines **Related:** Chapter 19 - The Research Programs (the same falsification discipline, at system scale) **Memory:** `docs/memory/engine-memory.md`.

Part IV - Decision Making

Chapter 10 - The Four Scoring Engines

Part IV - Decision Making Status of this chapter: Written

Why this chapter exists

`EngineRunner` (invariant #2, Chapter 2) runs exactly four engines against every candle, unconditionally, and rejects the candle outright if any one is missing. This chapter is the canonical explanation of the other three - Gaussian, Zone Gate, RR - since CRT already owns Chapter 8. It's also where this book delivers its most important warning about the decision spine: three of the four engines have been formally audited and found to contribute far less signal than their names suggest.

What problem it solves

Explains what each engine actually computes (as opposed to what its name implies), and gives each its correctly scoped status - because getting this wrong is exactly how a system ends up trusting a number that's secretly closer to a constant.

What you need to already know

Chapter 7 (the 39-dim vector every engine consumes) and Chapter 8 (CRT, already covered).

The idea

CRT - see Chapter 8

The CRT engine's score comes from `src/engines/crt_engine.py`, delegating to `engines/scoring_engine.py`. Its full explanation, including the state-machine/score-wrapper distinction, is Chapter 8 - this chapter doesn't repeat it, per the book's Canonical Concept Rule.

Gaussian - a channel that has largely collapsed to a constant

Two implementations exist behind a config switch (`gaussian_impl`): `HeuristicGaussianEngine` (`src/engines/heuristic_gaussian_engine.py`, the live default, an EMA/momentum kernel) and `MLGaussianEngine` (`src/engines/ml_gaussian_engine.py`, a trained Naive-Bayes model, fail-open to 0.5 on any error). Both ultimately route through the central scorer, `src/config_layer/crt_gaussian_scorer.py`.

The audited finding here is stark: the live Gaussian channel is an **unparameterized kernel that degenerates to a near-constant score of roughly 0.8825.** All eleven entries in `gaussian_registry.json` lack `mu`/`sigma`, so the score-normalization code's defaults (0/1) fire even on a *successful* registry load - meaning no learned parameter reaches the score at all, not even a mistrained one. Compounding this, `tanh(momentum_score)` saturates on 93-100% of bars because of a since-registered dimensional-mix defect

upstream in the feature math, discarding the sign of momentum entirely. An ablation test (pinning the channel at its saturated value, at neutral 0.5, and at live) produced byte-identical decisions across all three settings on four crypto majors - proof the channel is currently non-pivotal on both an information axis and a level axis. The model artifact that trained these registry entries was also confirmed to have trained on contaminated labels (the same F-022 labeling artifact discussed in Chapter 17).

Zone Gate - a real geometric hard gate, honestly measured as not adding edge

`src/engines/zone_gate_engine.py` scores candle geometry against `models/zone_registry.json` (8 zones), fail-open on registry errors, via `_compute_soft_zone_score()`. Unlike Gaussian, this channel is not degenerate - it's a working geometric filter that passes roughly 85% of candles it evaluates. But a fusion-weight and threshold sweep (weight in {0, 0.2, 0.4, 0.6}, threshold in {0, 0.25, 0.5}) produced byte-identical trade sets across every combination - the zone channel is redundant with what the rest of fusion already decides, not under-weighted. A separate re-derivation of its stored win/loss labels through an honest forward-walk (rather than the contaminated F-022 labeling) found the same story at the label level: zero of the eight zones clear a positive expectancy honestly. None of this implies live risk - the gate is purely geometric and fails open - but it does mean tuning its weight or threshold is currently a lever with no effect to pull.

RR - not actually a reward:risk engine

`src/engines/rr_engine.py`'s own module header states, plainly: it is "formerly misnamed 'RR Engine'" - it does not compute forward-looking reward:risk at all. What it actually scores is candle-close-position *commitment*: `max(upper_body, lower_body)` against the candle's range, i.e. how decisively a candle closed toward one extreme. The book keeps its file/class name (`RREngine`) for traceability but flags the naming mismatch here so a reader doesn't go looking for reward:risk math in the wrong file - the actual reward:risk economics for a trade live entirely in `UltronRiskGate`.

A parallel line of investigation found a second, distinct issue with a now-disabled sibling component, `rr_fusion`: its confidence-gate math was mis-scaled for the dimensionality of its input (a 27-degree-of-freedom Mahalanobis form whose in-distribution confidence floor sits near 1.4e-6, far below its own 0.3 bypass threshold) - so severely that **100% of its own training data** failed to pass its gate. That component stays disabled in the active config; a follow-up shadow test on its underlying model (bypassing the broken gate) found apparent out-of-sample discrimination, but on labels later confirmed to be F-022-contaminated - leaving its true economic value formally indeterminate, not proven and not disproven.

Classification

Concept	Status
CRT score	Production - see Ch.08
Gaussian (`HeuristicGaussianEngine` / `MLGaussianEngine`)	Production (wired, live), **AUDITED: non-pivotal, near-constant**

Zone Gate	Production (wired, live), **AUDITED: redundant, no marginal edge**
RR / `RREngine` (candle-commitment score)	Production (wired, live), **AUDITED: correctly named commitment score, not RR**
`rr_fusion` (disabled sibling)	Built, **disabled**, economic value indeterminate

Authoritative sources

- `src/engines/heuristic_gaussian_engine.py`, `ml_gaussian_engine.py`, `src/config_layer/crt_gaussian_scorer.py`.
- `src/engines/zone_gate_engine.py`, `models/zone_registry.json`.
- `src/engines/rr_engine.py`.
- `docs/topics/scoring-engines.md` - the always-synced topic doc covering Gaussian/Zone-Gate/RR together.
- `docs/current-findings.md` - F-036 (Zone Gate redundancy), F-038 (RR-fusion mis-gate origin), F-041B (honest zone labels), F-044 (dof-aware gate diagnosis), F-045/F-059 (RR clean-label re-test), F-060 (Gaussian constant collapse), F-061 (dimensional-mix mechanism generalized).
- `docs/governance/gaussian_lineage_audit.md`, `zonegate_lineage_audit.md`, `rr_lineage_audit.md` - the full per-engine AUDITED closure artifacts (see Chapter 17 for what "AUDITED" means as a closure status).

Unresolved questions

- RR's clean-label re-test (F-059) reached a "keep-candidate, research-only" verdict - whether a future pass changes `rr_fusion`'s disabled status is an open governance decision, not something this book resolves.

Previous: Chapter 09 - Interpreters and the Pattern Contract | **Next:** Chapter 11 - Fusion **Related:** Chapter 08 - The CRT State Machine | Chapter 17 - Truth Maintenance **Memory:** `docs/memory/engine-memory.md`.

Chapter 11 - Fusion: Combining Independent Signals

Part IV - Decision Making Status of this chapter: Written

Why this chapter exists

Four engines each produce a number (Chapter 10). Something has to turn four numbers into one. This chapter covers that step - and the one place in the entire synchronous spine where a language model is allowed to touch a live decision, under tightly constrained conditions.

What problem it solves

Explains how Fusion combines scores, why a neural-model slot exists but does nothing yet, and exactly how narrow the LLM's involvement is - directly enforcing invariant #3 from Chapter 2 ("the LLM is advice, never a trigger").

What you need to already know

Chapter 10 - Fusion's primary input is the Gaussian channel, whose audited near-constant behavior matters directly to how much this step is really doing.

The idea

A layered scorer, not a weighted average

`src/core/fusion_engine.py`'s `FusionEngine.evaluate()` is structured as a sequence of layers rather than a flat weighted sum:

1. **Gaussian scorer as the primary anchor.** Given Chapter 10's finding that Gaussian has largely collapsed to a near-constant, this is worth sitting with: the layer Fusion leans on hardest is currently the least informative of the four.
2. **A neural-model slot** - reserved for TradeNet v2, which is **built but never wired in** (F-005). This is a permanent, intentional stub, not an oversight in progress; wiring it up is a separate, explicitly gated qualification protocol (`TN_QUAL_V1`), not something that happens by accident.
3. **An LLM gate as an uncertainty tie-breaker** - and only that. It fires exclusively in a narrow confidence band, default `[0.45, 0.65]`. Any signal that's clearly strong or clearly weak never reaches the LLM at all. This is the concrete mechanism behind invariant #3: the LLM never decides a clear case, only helps break a tie in an ambiguous one, and (per Chapter 2) falls back to a neutral score after repeated failures rather than blocking.

What Fusion is, honestly, right now

Given the state of the channels feeding it - Gaussian near-constant, Zone Gate redundant with the rest of fusion (Chapter 10), RR correctly understood as a commitment score rather than reward:risk - Fusion's practical job today is closer to "pass CRT's signal through, sanity-checked by a narrow LLM tie-breaker on ambiguous cases" than "genuinely synthesize four independent opinions." That's not a criticism of the design - the architecture is exactly right for a system meant to *add* independent signal as each one earns its keep (Authority Ladder, Chapter 17) - it's just the honest current state, which the Research Programs in Part VII are directly aimed at changing.

Classification

Concept	Status
`FusionEngine.evaluate()` layered structure	Production
Gaussian-as-anchor	Production, but anchor is itself AUDITED near-constant (Ch.10)
Neural-model slot (TradeNet v2)	**Partial** - built, unwired (F-005)
LLM tie-breaker gate	Production, narrow-band only, fail-open

Authoritative sources

- `src/core/fusion_engine.py` - `FusionEngine.evaluate()`.
- `docs/topics/fusion-decision.md` - the always-synced topic doc (covers Fusion and Decision together).
- `docs/current-findings.md` F-005 (TradeNet stub) and F-060 (Gaussian anchor's audited status).
- `docs/governance/tradenet_lineage_audit.md`, `docs/governance/tradenet_qualification_protocol.md`
- the wire-up path for the neural slot, if it's ever taken.
- `docs/architecture/goal.md` Section 3, invariant #3 - the LLM-as-advice rule this chapter's gate enforces.

Unresolved questions

None for this pass - Fusion's structure and its channels' statuses are both well-evidenced by Chapter 10's audits and the TradeNet lineage audit.

Previous: Chapter 10 - The Four Scoring Engines | **Next:** Chapter 12 - The Decision Engine **Related:** Chapter 21 - The AI Automation Agent (the other place an LLM touches this repository, entirely off the synchronous spine) **Memory:** `docs/memory/engine-memory.md`, `docs/memory/architecture-memory.md`.

Chapter 12 - The Decision Engine: Semantic Approval Only

Part IV - Decision Making Status of this chapter: Written

Why this chapter exists

Fusion (Chapter 11) produces one combined score. Something has to turn that score into a binary ACCEPT/REJECT. This chapter covers that module - and a genuinely interesting piece of the repository's history: a multi-year-old bug that made this exact module a structural no-op, found and fixed by re-deciding *who owns what*, not by tuning a threshold.

What problem it solves

Explains what the Decision Engine actually decides today (semantic validity, not economics), and why that split exists.

What you need to already know

Chapter 11 - the Decision Engine consumes Fusion's output directly.

The idea

What it decides

`src/core/decision_engine.py`'s `DecisionEngine` is described in its own source as "central execution authority - only this module decides execute vs reject." What it actually evaluates is purely semantic: is this a valid market opportunity, based on the fused score, the win-probability estimate, and zone validity? It uses a dynamically calibrated threshold (the 85th percentile of recent scores, clamped to `[0.45, 0.65]`) rather than a single fixed cutoff.

What it used to decide, and why that broke everything silently

Until it was fixed, the Decision Engine also carried a reward:risk gate - comparing an RR-like signal against a `rr_threshold` of 1.5. The problem: the value it was actually comparing was candle *polarity*, a quantity bounded in `[0.5, 1]` (this is the "commitment score" from Chapter 10's RR engine discussion - a value that structurally can never exceed 1). Compared against a threshold of 1.5, this gate rejected as `low_rr` on **every single candle**: across a 70,002-candle test corpus, the gated code path executed zero times.

How it was resolved - an ownership decision, not a threshold tweak

The fix (F-048, 2026-07-24) wasn't to adjust the threshold or fix the polarity/RR mismatch in place - it was to ask *which module should own reward:risk economics at all*, and answer it cleanly: the Decision Engine's RR gate was **removed**. The Decision Engine is now semantic-approval-only (score / win-probability / zone / weak-component checks); real, cost-taxed reward:risk economics (`min_rr_ratio`, computed after the Execution Planner has determined actual SL/TP geometry) are owned solely by `UltronRiskGate`. The old `rr_threshold` config key is retained but retired (kept for hash-neutrality, no longer read for a decision). Because no real production path had ever actually fed genuine economic RR into the Decision Engine - only test injectors had - the removal was verified byte-identical against the live spine's historical output.

This is a good worked example of the repository's own governance discipline in Part VI: a bug that had silently zeroed out a gate for years was resolved by clarifying architecture (who owns reward:risk) rather than papering over the symptom (retuning a threshold that was comparing the wrong quantities in the first place).

Classification

Concept	Status
`DecisionEngine.decide()`, semantic-approval-only	Production, **current** design (post F-048)
The old RR gate inside Decision Engine	**Removed** (F-048, 2026-07-24) - retained config key is retired, not read
Dynamic threshold calibration (85th percentile, clamped)	Production

Authoritative sources

- `src/core/decision_engine.py` - `DecisionEngine.decide()`.
- `docs/current-findings.md` F-048 - the full resolution, including the byte-identical parity proof.
- `docs/topics/fusion-decision.md` - the always-synced topic doc.
- Chapter 14 - Ultron Risk Gate - the module that now solely owns reward:risk economics.

Unresolved questions

None - F-048 is an unusually well-documented, closed piece of history with its own parity proof already on record.

Previous: Chapter 11 - Fusion | **Next:** Chapter 13 - The Execution Planner **Related:** Chapter 14 - Ultron Risk Gate | Chapter 16 - Config-First Doctrine and the Promotion Path (the governance discipline this fix exemplifies) **Memory:** `docs/memory/engine-memory.md`.

Part V - Execution

Chapter 13 - The Execution Planner: From Decision to Order Geometry

Part V - Execution Status of this chapter: Written

Why this chapter exists

An ACCEPT from the Decision Engine (Chapter 12) is still just an approval - it's not yet an order. Something has to turn "yes, trade this" into a concrete entry price, and classify *what kind* of trade this is. This chapter covers that step, and flags two live, unresolved issues that anyone touching this code should know about before assuming the geometry it produces is trustworthy.

What problem it solves

Explains what `ExecutionPlannerV1_2` actually computes - and, just as importantly, what it deliberately does *not* compute.

What you need to already know

Chapter 12 - the planner only runs on an ACCEPT decision.

The idea

A classifier, not a geometry engine

`src/config_layer/execution_planner.py`'s `ExecutionPlannerV1_2.plan()` is, at its core, an intent classifier plus a gate. It sorts an accepted setup into one of four intents - BREAKOUT, PULLBACK, LIQ_SWEEP, REVERSAL - derives an entry price appropriate to that intent, and delegates final approval to `GateIntelligence`. **Stop-loss, take-profit, and reward:risk are explicitly not computed here.** That geometry comes from a different module entirely: CRT's own `compute_crt_levels` (inside `crt_engine_v2.py`), which owns the SL/TP math because it's the module that understands the structural levels (sweep point, displacement extent) the stop and target are actually measured against.

An open architectural caveat: which config the planner's geometry actually sees

There is a documented split-brain in how `CRTConfig` gets resolved on the *programmatic* path (tests, the tuner, embedders - anywhere `BacktestRunner(cfg)` is called with `crt_config=None`): in that case, `ConfigBuilder.build()` runs with no overrides, and the router's hardcoded FOREX/CRYPTO default profiles win - the production JSON config is never actually opened. Concretely, `expansion_atr_min_distance` ends up at `0.08` on this path versus `0.30` in the production JSON - a 3.75x difference on the very gate that governs the DISPLACEMENT->EXPANSION transition (Chapter 8's state machine). The CLI entry point is unaffected - this is specifically a programmatic-construction gap. It's open,

not yet fixed, and worth knowing about before trusting a programmatically-run backtest's structural-gate behavior at face value.

A second open caveat: SL geometry may be using the wrong units

A separate, still-unresolved issue concerns `compute_crt_levels` itself: it appears to treat the `atr` feature as if it were already in price units when computing the stop-loss buffer, but - per Chapter 7's feature pipeline - `atr` in the canonical vector is close-relative, not an absolute price quantity. On XAUUSD this produces stop-loss buffers on the order of thousands of times too small (sub-cent stops on LIQ_SWEEP entries), and the same unit confusion is implicated in unbounded position sizing downstream. This is a real, identified, **currently unfixed** issue awaiting an explicit user decision on remediation - it is not a hypothetical, and it is not something this book fixes as a side effect of documenting it.

Classification

Concept	Status
`ExecutionPlannerV1_2.plan()` intent classification	Production
SL/TP geometry (`compute_crt_levels`, lives in CRT engine)	Production, **known defect, unfixed** (ATR unit mismatch)
Programmatic `CRTConfig` resolution (non-CLI paths)	Production, **known split-brain, OPEN** (F-057)

Authoritative sources

- `src/config_layer/execution_planner.py` - `ExecutionPlannerV1_2.plan()`.
- `src/config_layer/crt_engine_v2.py` - `compute_crt_levels` (the SL/TP/RR geometry, owned by CRT not the planner).
- `docs/topics/execution-planning.md` - the always-synced topic doc.
- `docs/current-findings.md` F-057 - the `CRTConfig` programmatic split-brain, in full.
- `docs/topics/ai-automation-agent.md` Section GateIntelligence cross-references (if present) - the approval delegate.

Unresolved questions

- **The ATR unit mismatch in `compute_crt_levels`** - identified, scoped, and awaiting a user decision on remediation; this book does not resolve it. Rolled up in A2.
- **F-057's `CRTConfig` split-brain** - status OPEN per the finding itself; the fix is scoped as "backtest Phase 2" but not yet implemented as of this writing.

Previous: Chapter 12 - The Decision Engine | **Next:** Chapter 14 - Ultron Risk Gate **Related:** Chapter 08 - The CRT State Machine (owns the SL/TP math this chapter describes) **Memory:** `docs/memory/architecture-memory.md`.

Chapter 14 - Ultron Risk Gate: the Final Capital Check

Part V - Execution Status of this chapter: Written

Why this chapter exists

Everything before this point in the spine - ontology, features, four engines, fusion, a semantic decision, a planned entry - can be exactly right and the trade can still be wrong for the *portfolio*. This chapter covers the last, deterministic checkpoint before an order would leave the system, and the one module every other live-trading path in the repository is required to pass through.

What problem it solves

Separates "is this a valid opportunity" (Decision Engine, semantic) from "can we safely afford this trade right now" (Ultron, capital/portfolio). Chapter 12 already explained why that separation exists structurally (F-048); this chapter covers the module that resulted from it.

What you need to already know

Chapter 12 - specifically, why reward:risk economics live here and not in the Decision Engine.

The idea

What it checks

`src/core/ultron_risk_gate.py`'s `UltronRiskGate.evaluate()` is the final capital-protection layer: position sizing, drawdown caps, exposure limits, daily-trade limits, and correlation thresholds across open positions. It is deterministic and hard-rejects on any breach - its own documentation is explicit that **no fallback path is permitted** here. Where the LLM gate in Fusion (Chapter 11) is allowed to fail open to a neutral score, Ultron is not allowed to fail open at all; a breach is a breach.

Where reward:risk economics actually live

Per Chapter 12's account of F-048: real, cost-taxed reward:risk economics (`min_rr_ratio`, computed after the Execution Planner has determined actual SL/TP geometry) are owned solely here, not in the Decision Engine. This makes Ultron the single place in the spine where "is this trade good enough, economically, net of costs" is actually answered.

The one module INOUT touches

Of the four async "kitchen feeders" from Chapter 2, INOUT (the parallel live-execution strategy rail, currently archived - see Chapter 15) is unusual: it doesn't feed the spine asynchronously the way Governance or Training do. It joins the *synchronous* spine directly, but only at this one point - `UltronRiskGate.evaluate()`

- meaning whatever INOUT proposes still has to clear the exact same capital-safety check as the main CRT spine. It is explicitly not one of the four `EXPECTED_ENGINES`; it's a second decision source feeding the same final gate.

Classification

Concept	Status
`UltronRiskGate.evaluate()`	Production - the terminal approve/reject authority
Reward:risk ownership (`min_rr_ratio`, cost-taxed)	Production, here since F-048
Shared join-point with INOUT	Production mechanism; INOUT itself is archived (see Ch.15)

Authoritative sources

- `src/core/ultron_risk_gate.py` - `UltronRiskGate.evaluate()`.
- `src/core/_wrapper.py` - the wrapper referenced by the topic doc's entry-point trace.
- `docs/topics/ultron-risk-gate.md` - the always-synced topic doc.
- `docs/architecture/signal-flow.md` Section 2.4 - the INOUT join-point documentation.
- `docs/current-findings.md` F-048 - reward:risk ownership.

Unresolved questions

None for this chapter - Ultron's own scope is unusually crisply documented (a hard-reject-only module by explicit design), and its relationship to F-048 and to INOUT is already covered precisely by other chapters this one links to.

Previous: Chapter 13 - The Execution Planner | **Next:** Chapter 15 - Live Execution and INOUT **Related:** Chapter 12 - The Decision Engine **Memory:** `docs/memory/architecture-memory.md`.

Chapter 15 - Live Execution and INOUT

Part V - Execution Status of this chapter: Written

Why this chapter exists

Everything in Parts II-V so far describes the backtest path faithfully, and the live path *mostly* the same way - but "mostly" is doing real work in that sentence, and this chapter is where the book is most honest about the gap between the two. It closes Part V by describing what actually runs on a live tick, and by flagging rather than resolving a genuine ambiguity this research pass surfaced about the state of the parallel INOUT rail.

What problem it solves

Distinguishes "what the backtest replays" from "what a live tick actually triggers," and is explicit about which of the two the rest of this book's Parts II-V chapters were describing (mostly the former, since it's the better-evidenced, more heavily tested path).

What you need to already know

Chapter 14 - the one point where a live-execution rail rejoins the synchronous spine.

The idea

The live tick entry point

The confirmed entry point for a live candle is `HookedLiveEngine.process` in `src/runtime/live_engine_hook.py:582`. This is what actually gets called on a live tick, and it draws on `src/live/`, `src/inout/`, and `src/execution/loop.py` as supporting modules.

`src/execution/loop.py` - real code, no caller yet

`src/execution/loop.py` (plus `alert_manager.py`, `override_handler.py`) implements a multi-signal execution loop, but per the repository's own topic tracking it is explicitly **scaffolding - no caller yet**. It has tests (`test_execution_loop`) but isn't wired into a live path that actually invokes it.

INOUT - a genuine open question this book surfaces rather than resolves

Two things are both true and, on their face, appear to be in tension:

- `docs/architecture/goal.md` describes INOUT (a parallel live-execution strategy rail - scanner -> state machine -> controller) as **currently archived**, living at `archive/inout_legacy/`, joining the synchronous spine only at `UltronRiskGate.evaluate()` when active.
- `src/inout/` also exists as a live, non-archived subpackage (11 files) that CLAUDE.md's own file-placement convention (Section 3.1) still names as the destination for "live-mode code."

This book did not verify, this pass, whether `src/inout/` is the *current* successor to the archived `archive/inout_legacy/` rail, a different in-progress component with an overlapping name, or a remnant not yet cleaned up. Rather than guess, that's recorded honestly below as an unresolved question - exactly the discipline Chapter 3 committed this book to.

Classification

Concept	Status
`HookedLiveEngine.process` (live tick entry point)	Production
`src/execution/loop.py` (multi-signal execution loop)	**Partial** - built, tested, no live caller
INOUT strategy rail (`archive/inout_legacy/`)	**Legacy / Archived** per `goal.md`
`src/inout/` (current subpackage)	**Unknown relationship to the above** - see below

Authoritative sources

- `src/runtime/live_engine_hook.py:582` - `HookedLiveEngine.process`.
- `src/execution/loop.py`, `alert_manager.py`, `override_handler.py`.
- `docs/topics/live-execution.md`, `docs/topics/execution-loop.md` - the always-synced topic docs.
- `docs/architecture/goal.md` Section 2 - the "INOUT currently archived" statement.
- `src/inout/` - the current subpackage (11 files), whose relationship to `archive/inout_legacy/` is this chapter's open question.

Unresolved questions

- **Is `src/inout/` the live successor to the archived INOUT rail, a separate component, or dead code awaiting cleanup?** Not resolved this pass - needs a direct read of both `src/inout/`'s current contents and `archive/inout_legacy/` to answer definitively. This is the single most concrete "go verify this" item this book produced; rolled up in A2.

Previous: Chapter 14 - Ultron Risk Gate | **Next:** Chapter 16 - Config-First Doctrine and the Promotion Path
Related: Chapter 02 - The Invariants and the Happy Flow (where INOUT was first introduced as a kitchen feeder) **Memory:** `docs/memory/runtime-memory.md`.

Part VI - Governance

Chapter 16 - Config-First Doctrine and the Promotion Path

Part VI - Governance Status of this chapter: Written

Why this chapter exists

Parts II-V described the engine as it exists at one point in time. This chapter starts Part VI by explaining how that engine is meant to *change over time* - through configuration, under a formal gate - rather than through direct code edits chasing whatever the latest backtest suggested.

What problem it solves

Answers two related questions: what's allowed to be a config knob versus what has to stay in code, and what a config has to prove before it's allowed to become the one running in production.

What you need to already know

Chapter 2's invariant #4 (no config reaches production without governance) - this chapter is that invariant's full mechanism.

The idea

Config-First: the target end-state

The doctrine's stated target is **"Frozen Engine + Mutable Behavior + Governed Evolution."** Code changes should become rare; config changes should become routine; and the system should improve by *generating configs* - sweeps, tuning, promotion - rather than by rewriting engines. Every constant in the codebase is meant to be classified into one of three tiers before it's touched:

Class	Examples	What happens to it
STRUCTURAL	interfaces, enum members, the CRT transition graph, execution order, vector dimensions	Stays frozen in code - changing it is expected to be rare and expensive
BEHAVIORAL	thresholds, weights, percentiles, clamps, tier boundaries	Externalized into a config section
GOAL-SEEKING	candidate-config generation, sweeps, promotion itself	Generated *as* config; the engine underneath stays untouched

A knob climbs a maturity ladder - `HARD_CODED -> CONFIG_WIRED -> CONFIG_DRIVEN` - with goal-seeking (automated search) as an orthogonal, optional further step, not a requirement for every knob.

The hard rule: no silent defaults

New behavioral knobs must be introduced through a strict `_require()` boundary - a missing config key is an **error**, never a soft `.get(key, some_literal)` fallback. This closes a specific, previously-real failure class where an active production config quietly lagged the code schema and ran on undocumented defaults without anyone noticing. Every migration to config additionally needs a parity proof: the new JSON value must equal the prior hardcoded literal, read strictly, producing a byte-identical backtest ledger - any drift means the JSON default was wrong, and the determinism gate catches it.

The Authority Ladder - tunable is not the same as trusted

A knob becoming config-driven grants **tunability**, never **authority**. The doctrine draws four distinct rungs, and conflating them is exactly the mistake this ladder exists to prevent:

```
Information exists -> Economic usefulness exists -> Authority earned -> Architecture
justified
```

A statistically detectable signal is not automatically economically useful; an economically useful signal doesn't automatically earn production/fusion weight; and even a successful consumer of a signal doesn't automatically justify adding more architectural complexity around it. Only a *demonstrated* improvement to the system's own economic goal metric (`G001`, Chapter 1) earns a rung. This is the same discipline behind Chapter 9's Interpreter Contract and Chapter 19's research falsification programs, stated here at the level of a general rule.

The promotion path itself

A new config's journey to production is a fixed pipeline:

```
AutoTuner -> results/tuner/checkpoint_multi.json -> ConfigValidator.validate(params,
csv_paths, config_id) - runs a per-instrument backtest against quality gates: HARD: min
trade count, max drawdown SOFT: win rate, expectancy, cross-instrument variance ->
ValidationReport{decision: APPROVE | REJECT} REJECT -> results/validation/rejected/
APPROVE -> results/validation/approved/ -> PromotionManager.promote_from_report() -
computes a SHA-256 config hash - archives the currently-active config - writes the new
configs/production/{version}.json - appends an immutable line to
configs/promotion_log.jsonl
```

Promotion refuses outright on a non-APPROVE decision, a duplicate version name, a hash-computation failure, or a write failure - and a write failure triggers an automatic rollback plus a `PROMOTION_FAILED` log entry, never a silent partial write. This is what makes `configs/production/ACTIVE_VERSION` (the Tier-0 runtime truth per `CLAUDE.md` Section 4.0) trustworthy: it can only ever point at something that passed this exact gate.

Classification

Concept	Status
Config-First Doctrine (classification tiers, maturity ladder, no-silent-defaults rule)	Production doctrine, actively enforced (`behavior_census.py` + test floor)
The Authority Ladder	Production doctrine, cited throughout Parts VI-VII
`PromotionManager` / promotion pipeline	Production
`ConfigValidator.validate()` hard/soft gates	Production

Authoritative sources

- `CLAUDE.md` Section 6.5 - the full Config-First Doctrine, including the Authority Ladder and the behavioral-vs-structural classification table.
- `docs/reference/governance.md` - the full `PromotionManager` API (`promote_from_report`, `promote_from_tuner_checkpoint`, `promote_direct`, `list_versions`, `load_version`, rollback procedure) - this chapter summarizes Section 1 of it; the rest is a direct reference, not duplicated here.
- `src/governance/promotion_manager.py`, `src/config_layer/config_validator.py`.
- `scripts/analysis/behavior_census.py` + `tests/test_behavior_census.py` - the enforcement contract pinning each migrated module's maturity so it can't silently drift back into code.
- `docs/topics/promotion-governance.md`, `docs/topics/config-validation.md` - the always-synced topic docs.

Unresolved questions

None - this is one of the most thoroughly, explicitly documented doctrines in the repository.

Previous: Chapter 15 - Live Execution and INOUT | **Next:** Chapter 17 - Truth Maintenance **Related:** Chapter 12 - The Decision Engine (F-048 as a worked Config-First-adjacent example) | Chapter 19 - The Research Programs (the Authority Ladder applied to research findings) **Memory:** `docs/memory/governance-memory.md`.

Chapter 17 - Truth Maintenance: Findings, Closure, and Authority

Part VI - Governance Status of this chapter: Written

Why this chapter exists

This chapter explains the machinery behind a vocabulary this book has already used in nearly every chapter - "AUDITED," "CLOSED," confidence levels, `F-0xx` finding IDs. Rather than define these piecemeal, this is their one canonical home: how the repository decides what it believes, how confident it is, and how it prevents a claim from quietly rotting into a false one.

What problem it solves

A repository this size, worked on across years by multiple humans and multiple LLMs, faces a specific failure mode: the same fact stated three different ways in three different docs, one of which is now wrong, with no way to tell which. This chapter covers the doctrine built specifically to prevent that - "silent truth divergence."

What you need to already know

Chapter 3 - this chapter fills in the classification vocabulary that chapter promised to explain in full.

The idea

The seven rules

The Repository Truth Maintenance Doctrine (`CLAUDE.md` Section 6.2) states seven rules, worth knowing as a set because they compound: (1) find the doc that already owns a topic before creating a new one; (2) classify every claim as `ALIGNED`, `DOC_DRIFT` (code wins, fix the doc), `CODE_DRIFT` (doc wins, fix the code), or `AMBIGUOUS`; (3) never silently resolve an `AMBIGUOUS` conflict - surface it and ask; (4) never delete disproven history - mark it `SUPERSEDED`/`INVALIDATED_BY` and keep the row; (5) minimize doc count; (6) synchronize related artifacts rather than editing in isolation; (7) state truth per-branch, never globally. Chapter 7's 38-vs-39-dim finding is a live, worked example of rule 2 in action - this book classified it and flagged it rather than silently patching it, per rule 3's caution and this chapter's own scope discipline.

Findings: the unit of validated belief

`docs/current-findings.md` is where a validated or overturned conclusion gets recorded, the same turn it's discovered. Every finding carries a confidence level - `Certain`, `Likely`, or `Possible` - and cites its evidence. Findings are never deleted, only marked `SUPERSEDED` or `RETIRED` with the row kept intact, so the repository's history of what it *used* to believe stays legible even after that belief changes. This book's own chapters cite specific findings (F-005, F-028, F-036, F-038, F-039, F-041B, F-044, F-048, F-057, F-060, F-062, and

others) exactly this way - as evidence, with the confidence and scope the finding itself declares, never inflated.

Epistemic Integrity - catching overclaims before they're registered

Before any finding is registered, a six-question pre-registration check runs: what artifact supports this (file:line required)? Could insufficient power explain the observation? Am I upgrading sign noise into meaning? Is this statistical or economic (a distinct authority tier)? Is the parent claim stronger than its children? Would I phrase this differently after seeing raw counts? If any answer exposes uncertainty, the finding's confidence gets downgraded or its status set to `HYPOTHESIS` rather than accepted as-is. When this check catches a real overclaim, the required phrasing is explicit and disarming rather than defensive: *"Caught me overclaiming; I owe you a correction."* A pre-registration correction under this ritual is treated as evidence the discipline worked, not as a failure - Chapter 10's Gaussian-audit ablation and this book's own 38-vs-39-dim flag in Chapter 7 are both examples of the same instinct: report the uncertainty rather than round it away.

Correction is fixing the source, not just the chat. If a false claim is retracted, the same turn also has to fix every recorded instance of it - the finding's `Reversal:` field, the doc, the memory file - marked `CORRECTED: <old> -> <new>`, never silently deleted.

Closure & Authority: what "CLOSED" and "AUDITED" actually mean

`docs/governance/closure_authority_index.json` tracks closure status per subsystem, governed by one non-negotiable invariant: **closure is boundary-scoped and non-transitive.** A `CLOSED` upstream subsystem does not make anything downstream of it `CLOSED` - only the specific artifact that declares closure for a *specific, named boundary* counts. Three distinctions worth memorizing, because conflating them is the exact mistake this index exists to prevent:

- `AUDITED != CLOSED` - reviewed and characterized is not the same as end-to-end proven.
- `LINEAGE CLOSED != ECONOMICALLY VALIDATED` - knowing a model loads the artifact it claims to is not the same as that artifact being worth anything.
- `UPSTREAM CLOSED != DOWNSTREAM CLOSED` - see the non-transitivity invariant above.

Concretely, in this book's own vocabulary (Chapter 3): CRT is `CLOSED` for its mechanism boundary only (Chapter 8); Gaussian, Zone Gate, RR, BitNet, and TradeNet are all `AUDITED` (Chapter 10, Chapter 11) - reviewed end-to-end, with clear, documented findings, but not certified as economically proven. A `CLOSED` or `AUDITED` surface reopens only under specific conditions: a governed dependency inside its declared boundary changes, a closure invariant fails, new contradictory evidence appears, or the artifact's own stated revalidation condition fires - never just because someone feels like revisiting it.

Classification

Concept	Status
Truth Maintenance Doctrine (7 rules)	Production doctrine, actively enforced (test-guarded)

Findings registry (`current-findings.md`)	Production, living document
Epistemic Integrity ritual (E-001)	Production doctrine
Closure & Authority Index	Production, machine-readable + test-guarded

Authoritative sources

- `CLAUDE.md` Section 6.2 (Truth Maintenance, the seven rules, the Documentation Drift Protocol, the Findings Mandate) and Section 6.5 (the Authority Ladder, cross-referenced from Chapter 16).
- `docs/current-findings.md` - the living findings registry.
- `docs/governance/EPISTEMIC_INTEGRITY.md` - the full six-question ritual.
- `docs/governance/closure_authority_index.json` - the machine-readable closure registry.
- `docs/governance/DOCUMENTATION_DRIFT_PROTOCOL.md` - the full drift-classification process.
- `tests/test_current_findings.py`, `tests/governance/test_epistemic_invariants.py`, `tests/test_closure_authority_index.py` - the enforcement floors.

Unresolved questions

None - this doctrine is unusually self-documenting; its own charter is the primary source and this chapter is a faithful compression of it.

Previous: Chapter 16 - Config-First Doctrine and the Promotion Path | **Next:** Chapter 18 - A Field Guide to docs/governance/ **Related:** Chapter 03 - How This Book Fits (the classification vocabulary this chapter grounds) **Memory:** `docs/memory/governance-memory.md`.

Chapter 18 - A Field Guide to docs/governance/

Part VI - Governance Status of this chapter: Orientation-level

Why this chapter exists

`docs/governance/` has roughly 250 files - by far the largest single documentation subtree in the repository. Chapters 16 and 17 already gave you the *doctrine* (Config-First, Truth Maintenance, Closure & Authority). This chapter is different in kind: it's a map of the directory itself, so that when you land in it - chasing a citation from another chapter, or investigating a specific subsystem's audit trail - you know which cluster you're in and what kind of document to expect.

What problem it solves

Without a map, `docs/governance/` reads as an undifferentiated pile of dated JSON+MD pairs. It isn't - it's several distinct evidence ledgers, each with its own internal logic, that happen to share a directory.

What you need to already know

Chapter 17 - nearly everything in this directory is either a closure artifact, a lineage audit, or a certification record in that chapter's sense.

The idea

This is mostly an evidence ledger, not narrative documentation

The distinction from Chapter 3 matters most here: most of `docs/governance/` is machine-oriented certification/evidence material - dated, often paired JSON+MD, produced by a specific audit or census run - not curated prose meant to be read start to finish. Treat this chapter as a directory of *drawers*, not a table of contents to read in order.

The clusters

- **Promotion / production lifecycle.** The mechanics are Chapter 16's territory; this directory holds supporting evidence like `IMPLEMENTATION_VALIDATION_V1.md`.
- **Closure reports.** End-of-thread declarations for a specific boundary - `crt_closure_report.md` (Chapter 8's primary source), `canonical_feature_code_surface_closure-2026-07-11.md`, various `feature_pipeline_*_closure_manifest` and `feature_surface_closure_audit` pairs.
- **Lineage audits.** Does the runtime actually load the artifact its registry claims to, and did the trainer actually produce it honestly - one file per model family: `gaussian_lineage_audit.md`, `zonegate_lineage_audit.md`, `rr_lineage_audit.md`, `bitnet_lineage_audit.md`, `tradenet_lineage_audit.md`, plus a cross-model `model_lineage_rollup.md`. These are Chapter 10 and Chapter 11's primary evidence.

- **Epistemic integrity / adjudication.** `EPISTEMIC_INTEGRITY.md` (Chapter 17's primary source) plus dated adjudication records where two or more authorities disagreed and the disagreement had to be formally resolved (e.g. `three_authority_drift_adjudication-2026-07-11.md`, `geometry_semantic_adjudication.jsonl`).
- **Ontology / market-ontology contracts.** `MARKET_ONTOLOGY_EVOLUTION_CONTRACT.md` (Chapter 6's primary source), plus formula- and geometry-specific contracts (`crt_formula_contract.md/json`, `geometry_family_registry.json`).
- **Measurement contract.** `MEASUREMENT_CONTRACT.md` and its schema - a frozen, not-yet-implemented program for making research claims formally comparable (see Chapter 19).
- **Feature-DAG / FM certification.** A large cluster tracking the certification status of individual feature-math (`FM-0NN`) nodes as they climb the ontology's refinement ladder (Chapter 6) - `feature_dag_layers-*`, `feature_certification_ledger.jsonl`, per-feature certification files.
- **CRT-specific census/diagnostic artifacts.** `crt_executable_surface.md/json`, `crt_executable_state_graph.md/json`, `CRT_CONFIG_CONSTRUCTION_PROTOCOL.md`, and dated funnel/fail-reason diagnostics - deep supporting evidence for Chapter 8.
- **OHLCV corpus authority.** A dated 2026-07-10 cluster establishing exactly what the raw candle corpus is, its lineage, and its temporal semantics - `CORPUS_AUTHORITY.md` plus census/closure/contradiction/fingerprint artifacts. Supporting evidence for Chapter 5.
- **MSIP (Market State Interpretation Platform).** A shadow-design program with its own design contract and implementation plan (`MSIP_SHADOW_DESIGN_CONTRACT_V1.md` and siblings) - not yet covered by its own book chapter (see A2).
- **Script inventory / authority.** `script_canonical_allowlist.json`, `MODEL_INTENT_AUTHORITY_REGISTER.md` - governs which scripts are recognized, registered surfaces (referenced by `CLAUDE.md` Section 3.1's SITS registration requirement).
- **Documentation governance itself.** `DOCUMENTATION_DRIFT_PROTOCOL.md` (Chapter 17's primary source), plus `finding_dependency_audit.md` and `user-progress-registry.md`.
- **Ten packaged decision-bundle subdirectories** (`build_manifests/`, `crt_architecture_adjudication_v1/`, `msip_shadow_design_v1/`, `rr_l1_freeze/`, `xauusd_crt_baseline_trace/`, and others) - self-contained evidence packages for a specific past decision, not meant to be browsed loose.

How to actually use this directory

Don't browse it top-down. Arrive with a specific question - "is Gaussian's model artifact actually loaded correctly," "what does the CRT closure boundary formally cover," "why is `rr_fusion` disabled" - and use Chapter 17's Closure & Authority Index or `docs/current-findings.md`'s `Evidence:` fields to find the specific file. This directory rewards targeted lookup, not linear reading.

Classification

Concept	Status

`docs/governance/` as a whole	Production evidence ledger - actively maintained, test-guarded per-cluster
Individual clusters	Mixed - closure reports and lineage audits are authoritative for their narrow boundary; census/diagnostic artifacts are point-in-time evidence, not living truth

Authoritative sources

- The clusters listed above, by name, are the entry points - there is no single index file for this directory beyond `docs/current-findings.md`'s `Evidence:` cross-references and the Closure & Authority Index in Chapter 17.

Unresolved questions

- **MSIP** doesn't yet have its own book chapter - it's a large enough program (its own design contract, implementation plan, and preregistration series in `docs/research-readiness/`) to arguably deserve one. Deferred to a future pass; tracked in A2.
- This chapter's cluster list is a first-pass grouping, not an exhaustive index of all ~250 files - a future deepening pass could build a proper searchable index if that turns out to be worth the maintenance cost (weighed against `CLAUDE.md` Section 6.2 rule 5's "minimize doc count").

Previous: Chapter 17 - Truth Maintenance | **Next:** Chapter 19 - The Research Programs **Related:** Chapter 06, Chapter 08, Chapter 10 (the chapters whose primary evidence lives in this directory) **Memory:** `docs/memory/governance-memory.md`.

Part VII - Research

Chapter 19 - The Research Programs: Falsification as a Discipline

Part VII - Research Status of this chapter: Orientation-level

Why this chapter exists

Chapter 1 promised that pattern interpreters and, by extension, trading ideas in general are "measured, never asserted." This chapter is where that promise gets tested at scale: a numbered sequence of Research Programs, each pre-registered, each run to a decisive conclusion, and - almost without exception so far - each one closing as an honest null result. That's not a failure of the research effort. It's the effort working exactly as designed.

What problem it solves

Gives a reader the shape of the research history without re-deriving nine programs' worth of statistics: what was tested, what the verdict was, and why "0 PROMOTE" across program after program is a *credible* finding rather than a broken research pipeline.

What you need to already know

Chapter 16's Authority Ladder - every program below is that ladder applied to a specific hypothesis: information existing is not the same as economic value, which is not the same as earned authority.

The idea

The gate every program runs through

Nearly every program in this ledger is tested against the same standard: a pre-registered hypothesis, an intrabar-realistic exit model with real transaction costs (not close-only fills), and a decision rule that requires *absolute* expectancy above zero out-of-sample - not just beating a control. A program is only allowed to "PROMOTE" a finding into production authority if it clears that bar; anything else is `REJECT`, `INSUFFICIENT_POWER`, or `REDUNDANT` (statistically real but fully explained by something already known).

The ladder, program by program

- **Program 1 - Next-Bar Directional Ontology.** The foundational sweep: is there *any* directional edge in next-bar entries across crypto majors, at any conditioning level (session, volatility, momentum)? Ran through qualification, conditional-entropy pocket search, a selection-effect isolation, a full trade-anatomy audit, and an exit/cost grid - every stage closed null. **CLOSED / KILLED.** This is the foundation the rest of the ladder builds on.

- **Program 2 - Structural Asymmetry.** Does a *completed* sweep->displacement->retest sequence add forward asymmetry beyond a bare sweep? One experiment, a negative point estimate, badly underpowered (n~47, roughly 1% funnel completion). **FROZEN after one experiment** - explicitly not run further until power improves.
- **Program 3 - Higher-Timeframe Directional Ontology.** Does moving to H1/H4 rescue the directional edge Program 1 killed at M15? No - zero promotions at either timeframe. **CLOSED.**
- **Program 4 / 4b - Volatility Regime and Transition.** Does contemporaneous volatility-regime *level*, or a forward Markov *transition* forecast, offer a consumable directional edge? Both informative (volatility genuinely has memory) but neither consumable - the transition-forecast variant (4b) turned out to be statistically redundant with plain current-volatility level, adding nothing beyond what was already known. **CLOSED**, both variants.
- **Program 5 - Cross-Sectional Dispersion.** The first program to leave the per-instrument frame entirely and ask whether relative-value/market-neutral positioning across instruments is monetizable. All five interpreters tested REJECT, well-powered. **KILLED.**
- **Program 6 / 6b - Carry / Basis and Carry Harvest.** Is funding-rate carry a usable signal (6), or - separately - is the raw cash-and-carry payoff itself harvestable net of costs (6b)? Both null: all eight carry/basis interpreters REJECT, and all four harvest constructions fail to clear their own turnover cost even before considering price drag. **Both KILLED**, with an explicit stop called after 6b rather than continuing to iterate on cost assumptions.
- **Program 7 - Open Interest.** Named as explicitly out of scope for Program 6 (a distinct payoff), but not yet run as of the evidence this chapter draws on.
- **Program 8 - Weekly Liquidity-Sweep Ontology.** A genuinely new calendar-locked geometry (weekly accumulation/sweep/reversal, distinct from Program 1/2's local-structural frame) tested on FX majors. Zero promotions across all six scopes tested. **CLOSED.**
- **A cross-asset generalization check (not itself numbered as a Program):** does Program 1's crypto-majors null generalize to FX majors? Yes - the same entry-information null replicated on five FX majors with well-powered, decisively negative toy results.

Why this is the right outcome, not a broken pipeline

Every one of these closures follows the same discipline as Chapter 9's Interpreter Contract: pre-register the test, apply a fixed, honest cost/exit model, require *absolute* expectancy, and accept the answer even when it's negative. A research program that only ever produced PROMOTE verdicts on this codebase's own history would be the actual red flag - it would mean the bar wasn't being held. Programs 1 through 8 closing null, one after another, under the same rigorous standard, is what a functioning falsification discipline looks like from the outside.

Classification

Concept	Status

Programs 1, 2, 3, 4, 4b, 5, 6, 6b, 8	Research - all **CLOSED/KILLED/FROZEN**, null or insufficient-power results
Program 7 (Open Interest)	Not yet run
FX cross-asset generalization of Program 1's null	Research - confirmed generalizes

Authoritative sources

- `docs/current-findings.md` - the Funding Ledger section, Programs 1 through 8, each with its own `F-0xx` findings and confidence levels - this chapter's primary source, summarized not reproduced.
- `docs/research-readiness/program-*-preregistration.md` - the pre-registration for each program (e.g. `program-4-nondirectional-preregistration.md`, `program-8-weekly-crt-sweep-preregistration.md`).
- `CLAUDE.md` Section 6.5 - the Authority Ladder this whole chapter is an application of.

Unresolved questions

- **Program 9** appears in recent repository commit history ("Program 9 M1 - M5 ladder + OCO straddle kernel", "Program 9 M2/M4 drivers") but was not present in the Funding Ledger structure this chapter's research verified - its scope, hypothesis, and current status are not covered here and should be treated as **not yet documented in this book**, not as absent from the repository. Tracked in A2 as the most likely next program to chapter.
- Program 7 (Open Interest) is named but its current status (planned, in progress, or not started) wasn't independently verified this pass.

Previous: Chapter 18 - A Field Guide to docs/governance/ | **Next:** Chapter 20 - The Research Platform
Related: Chapter 09 - Interpreters and the Pattern Contract (the same discipline, one interpreter at a time) | Chapter 16 - Config-First Doctrine (the Authority Ladder) **Memory:** none dedicated - see the research-family memory entries indexed in `MEMORY.md` under this session's project files (e.g. cross-sectional, carry-basis, carry-harvest programs).

Chapter 20 - The Research Platform: src/research/

Part VII - Research Status of this chapter: Orientation-level

Why this chapter exists

Chapter 19 covered the *findings* the research effort produced. This chapter covers the *machinery* that produces them: `src/research/`, the single largest subpackage in the entire codebase (416 files - more than double the next largest), plus its companion pre-registration corpus in `docs/research-readiness/`.

What problem it solves

Explains why there's so much code here, what shape it takes, and - importantly - flags a real gap between the research platform's ambition and its current implementation state, so a reader doesn't assume more rigor is already enforced than actually is.

What you need to already know

Chapter 19 - this chapter is that one's supporting infrastructure.

The idea

The shape of `src/research/`

Structured as ~17 subpackages plus loose top-level modules. The subpackage names roughly track the research programs and techniques from Chapter 19 and beyond: `adapters/`, `candle_state/`, `clean_labels/`, `controls/` (the negative controls every program is tested against), `envelope_offline/`, `episodes/`, `hypotheses/`, `ic002_entry_evolution/`, `ic003_shapes/` and `ic003b_sequence_geometry/` (interpreter-candidate shape research), `measurement/`, `model_runners/`, `path/`, `secondlow_v1/`, `synthetic/`, `weekly_sweep/` (Program 8's home), `zone_mapping/`. The loose top-level modules include the shared machinery every program reuses: `cross_sectional.py` (Programs 5/6/6b's kernel), `exit_grid.py` and `costs.py` (the honest-cost exit model every program is judged against), `conditional_entropy_grid.py`, `experiment_spec.py`, `goal_alignment.py`, `forensics.py`.

The Edge Research Platform (ERP)

`docs/research-readiness/` documents a broader design - the "Edge Research Platform" - covering how research should flow end-to-end: an information-class boundary, a phased plan, a "promise ladder" (explicitly the doctrine-side counterpart of the Authority Ladder from Chapter 16), and how multiple LLMs collaborate on research work (tying into Chapter 23). This directory also holds every individual program's pre-registration document (`program-N-*-preregistration.md`) and a series of individual hypothesis pre-registrations (`h-*` files) for smaller, single-hypothesis tests that don't rise to full-Program scale.

An honest gap: the Research Measurement Contract exists on paper, not yet in code

One specific piece is worth flagging precisely because it's easy to miss: `docs/governance/MEASUREMENT_CONTRACT.md` defines a frozen (as of 2026-07-10) schema meant to make every research claim's measurement basis - label derivation, cost model, gate mode, clock basis - explicit and comparable across experiments, backed by a 27-seed adversarial mutation test matrix. As of the most recent verification available to this book, **zero `MC-*` instances have been sealed and none of the 27 probes have been implemented** - every finding in `docs/current-findings.md` currently carries `Contract: UNKNOWN`. This does not invalidate the Program 1-8 findings in Chapter 19 - they were run under their own program-specific rigor (pre-registration, honest costs, absolute-expectancy bar) - but it does mean the *cross-program comparability* layer the Measurement Contract is meant to provide doesn't exist yet. Anyone tempted to compare two programs' results side by side should treat that comparison as informal until this layer is built.

Reports that check the platform's own hygiene

`docs/research-readiness/` also holds a set of integrity reports that audit the research *infrastructure* itself rather than any one finding: `determinism-report.md` (is the same experiment reproducible), `metric-integrity-report.md`, `config-reachability-report.md`, `test-gap-report.md`, `behavior-census-report.md`. These exist so that a null result can be trusted as a genuine null, rather than an artifact of broken plumbing.

Classification

Concept	Status
`src/research/` subpackages	Research infrastructure, actively used (Programs 1-8)
ERP design docs (`docs/research-readiness/edge-research-platform-*`)	Design doctrine, partially implemented
Research Measurement Contract	**Frozen schema, unimplemented** - 0/27 probes built, all findings `Contract: UNKNOWN`
Research-readiness integrity reports	Production tooling (self-audits the research platform)

Authoritative sources

- `src/research/` - the 17 subpackages and shared modules listed above.
- `docs/research-readiness/README.md` and its ~60 sibling files (ERP design series, program and hypothesis pre-registrations, integrity reports).
- `docs/governance/MEASUREMENT_CONTRACT.md` - the frozen, unimplemented contract schema.
- `docs/topics/research-measurement-contract.md` - the always-synced topic doc for this specific gap.

Unresolved questions

- Whether any `MC-*` instance work has progressed since the Measurement Contract's last verified state (`MEASUREMENT_LAYER_STATUS=OPEN`, 0 sealed) was not re-checked in this pass - verify against `docs/governance/research_family_registry.json` directly before relying on this chapter's snapshot.
- This chapter did not individually verify every one of the 17 `src/research/` subpackages' current wiring status (research-only vs. touching any promotable artifact) - flagged in A2.

Previous: Chapter 19 - The Research Programs | **Next:** Chapter 21 - The AI Automation Agent **Related:** Chapter 23 - Multi-LLM Coordination (the ERP's multi-LLM research-collaboration design) **Memory:** the session memory entry on the research family registry + measurement profiles (title: "Research family registry + measurement profiles") records the Measurement Contract's frozen-since-2026-07-10, zero-implementation status - consult it via the assistant's memory index, not a repo-relative path (it lives outside this repository).

Part VIII - Agent Intelligence

Chapter 21 - The AI Automation Agent

Part VIII - Agent Intelligence Status of this chapter: Orientation-level

Why this chapter exists

Everything in Parts II-VII describes a system a human (or a script) drives directly. This chapter covers the layer that lets natural language drive it instead - a deterministic, auditable agent built specifically to avoid the failure mode of "the LLM improvised something." Keep this distinct from Chapter 23: that chapter covers *humans and multiple LLMs collaborating on developing this repository*; this chapter covers *one in-repo agent operating the trading system itself* through a fixed set of tools.

What problem it solves

Explains how a natural-language command becomes a specific, bounded sequence of tool calls against the trading system - and why that sequence is looked up, not generated fresh, on every request.

What you need to already know

Chapter 2 - the agent is one of the four async kitchen feeders, and per invariant #3 it can drive the spine (Pipeline mode) but never silently mutate a decision in flight (Copilot mode is explicitly read-only).

The idea

Five modes, each with its own tool surface

`src/agent/modes/` implements five distinct operating modes: **Pipeline** (drives the spine through a backtest loop - 6 tools), **Copilot** (taps the Fusion/Decision step read-only, cannot mutate - 7 tools, all read-only by construction), **Governance** (interacts with the promotion machinery from Chapter 16 - 5 tools), **Findings**, and **Log Query**. Two additional tools are available cross-mode. This mode separation is itself a safety mechanism: a Copilot-mode session is structurally incapable of triggering a trade or a promotion, regardless of what it's asked to do, because the tools that could do that simply aren't in its registry.

Deterministic planning, not generated planning

This is the agent's central design decision, and it's worth dwelling on: when a natural-language command comes in, an `IntentRouter` classifies it (a regex-first pass, with an LLM fallback for ambiguous phrasing) into one of a fixed catalogue of intents. A `PlanCompiler` then looks that intent key up in `PLAN_REGISTRY` - a hardcoded dictionary in `src/agent/plan_compiler.py`, "the single source of truth for what steps each intent runs." **The LLM never generates the tool-call sequence itself.** It only helps identify *which* pre-written sequence applies. This is the same "LLM is advice, never a trigger" invariant from Chapter 2, applied at the level of *what the agent is allowed to do* rather than just *what decision it can influence*.

An `Executor` then runs the compiled plan, gated by a confirm-step and a path-guard (preventing the agent from writing outside its authorized surface), tracked through `AgentState`.

Everything is logged, twice

Every agent action writes to `logs/agent_audit.jsonl` (a per-step record of exactly what tool ran with what arguments) and `logs/agent_intent_log.jsonl` (a session-level summary of what was asked and how it was classified). This gives the same replayability guarantee (Chapter 2's invariant #1) to agent-driven work that the core spine already has for candle-driven work.

Classification

Concept	Status
Five-mode structure (Pipeline/Copilot/Governance/Findings/Log Query)	Production
`PLAN_REGISTRY` deterministic lookup	Production - the core safety mechanism
`IntentRouter` (regex + LLM fallback)	Production
Dual audit logging	Production

Authoritative sources

- `docs/reference/agent-reference.md` - the complete reference: all tools per mode, the intent catalogue, the full `PLAN_REGISTRY` tables, the `Executor`'s confirm-gate and path-guard, audit log field semantics, and the procedure for adding a new tool. This chapter summarizes it; that document is the one to read for implementation detail.
- `src/agent/plan_compiler.py` - `PLAN_REGISTRY` (line ~43) and `PlanCompiler.build()`.
- `src/agent/intent_router.py`, `tool_registry.py`, `executor.py`, `state.py`, `audit.py`.
- `src/agent/modes/` - the five mode implementations.
- `docs/topics/ai-automation-agent.md` - the always-synced topic doc.

Unresolved questions

None for the architecture itself - `agent-reference.md` is a comprehensive, well-structured primary source. Whether every documented tool in the reference is currently reachable end-to-end (vs. scaffolded) wasn't independently re-verified this pass.

Previous: Chapter 20 - The Research Platform | **Next:** Chapter 22 - The Control Plane **Related:** Chapter 23 - Multi-LLM Coordination (a different, human-orchestrated layer - don't conflate the two) **Memory:**

`docs/memory/agent-memory.md`.

Chapter 22 - The Control Plane

Part VIII - Agent Intelligence Status of this chapter: Orientation-level

Why this chapter exists

The repository has no REST API by design (Chapter 2's invariant #7 - no database, no broker, no cloud dependency extends to "no network service surface" as a default posture). But someone still needs a single, discoverable catalogue of every command this system supports, and a place for a human-facing dashboard to hang off of. That's the control plane.

What problem it solves

Gives every CLI-invocable command in the system one canonical, machine-readable declaration - instead of a human needing to grep `scripts/` to discover what's runnable.

What you need to already know

Chapter 21 - the AI Agent's tool registry and the control plane's command registry are siblings in spirit (both are declarative catalogues), though distinct in scope (the agent's tools operate the trading system; the control plane catalogues *every* runnable command).

The idea

A declarative catalogue, not an execution engine

`src/control_plane/registry.py` defines a `CommandSpec` for every registered command (its `ArgSpec`s, from `cp_types.py`, and an argv template) - a catalogue, not the execution logic itself. It also carries a `WORKFLOW_STAGE_ORDER` tuple laying out the canonical operational pipeline in six stages: **Data Prep -> Tuning -> Model Training -> Validation & Promotion -> Replay & Backtest -> Live Runner** - and a mapping from roughly 19 concrete command IDs (`data.prepare_data`, `tuning.auto_tuner_multi`, `governance.orchestrator`, `promotion.manager`, `backtest.v2`, `live.inout_runner`, and others) to the stage they belong to, plus a small set of quickstart usage notes per command. This is effectively the machine-readable backbone behind `docs/reference/cli-matrix.md`'s human-readable command catalog.

The localhost dashboard

A small HTTP surface (`server.py`, `dashboard_api.py`, `report_api.py`) exposes this registry to a browser-based UI at `localhost:8787`, picking up new registered commands automatically via `ControlPlaneAPI.commands_payload()` - adding a `CommandSpec` is enough to make a command appear in the UI, with no separate UI-side wiring needed. This surface is explicitly **localhost-only, with no authentication layer and no TLS** - it is not meant to be exposed beyond the machine it runs on, and should never be.

Supporting modules

`code_context_extractor.py`, `context_report.py`, and `dot_graph_context.py` build the contextual information the AI Agent's Copilot mode (Chapter 21) can surface to an LLM - dependency-graph context, code excerpts - while `jobs.py` and `monitors.py` handle running and observing long-lived background operations the control plane kicks off.

Classification

Concept	Status
`CommandSpec` registry (`registry.py`, `cp_types.py`)	Production
Localhost dashboard (`server.py` + UI)	Production, deliberately unauthenticated/local-only
Context-extraction support modules	Production, primarily in service of the AI Agent's Copilot mode

Authoritative sources

- `src/control_plane/registry.py`, `cp_types.py` - the `CommandSpec` catalogue, `WORKFLOW_STAGE_ORDER`, and stage/command mapping.
- `docs/reference/control-plane.md`, `docs/reference/cli-matrix.md` - the full reference and the auto-generated command catalog.
- `src/control_plane/server.py`, `dashboard_api.py`, `report_api.py`, `jobs.py`, `monitors.py`.
- `CLAUDE.md` Section 4 - the localhost-only, no-auth constraint, stated as a hard operational limit.

Unresolved questions

None for this pass - the control plane's scope (declarative catalogue + localhost dashboard) is narrow and was confirmed directly against source.

Previous: Chapter 21 - The AI Automation Agent | **Next:** Chapter 23 - Multi-LLM Coordination **Related:** Chapter 04 - Architecture at a Glance **Memory:** none dedicated - see `docs/memory/architecture-memory.md` for cross-package context if needed.

Chapter 23 - Multi-LLM Coordination

Part VIII - Agent Intelligence Status of this chapter: Orientation-level

Why this chapter exists

This closes Part VIII with the one layer that isn't about the trading system at all - it's about *how this repository itself gets built*, when the work is spread across six different language models operated by hand by one human. Distinct from Chapter 21: that was one in-repo agent operating the trading system; this is six external LLMs collaborating on developing the repository, with the user as the bridge between them.

What problem it solves

Six models drifting into six different mental models of the same codebase is worse than one model working alone. This protocol exists to keep them sharing one reality.

What you need to already know

Nothing new - this chapter is largely self-contained, describing a process layer rather than code that runs inside the trading system.

The idea

The pipeline, and who's never allowed to skip it

Six models, each with a fixed role and an explicit "never does" boundary, connected in sequence by the user manually copy-pasting a structured handoff block between chat windows - no model calls another directly:

```
User (Bridge/Decider) -> DeepSeek (Planner - decomposes the story into a plan) -> Gemini Think (Quant Analyst - math/stats/CRT validation; never optimizes code) -> Gemini Pro (Optimizer - turns analysis into efficient code; never re-derives math) -> ChatGPT (Interpreter/Architect - explains, builds a JSON task plan; never executes tasks) -> Claude (Executor - writes code, runs validation, logs; never adds unsolicited explanation) -> Reality (tests + findings - VALIDATED / REFUTED / INCONCLUSIVE) -> Grok-2 (Synthesizer - combines everything + Reality into a final answer) -> User decides - approve, reject, or loop
```

Code authority is exclusive; advice is shared

This is the load-bearing rule of the whole layer: **Claude is the sole actor that writes or edits code and runs execution in this repository.** The other five models may propose code, diffs, or commands - but those are inputs *to* Claude, never applied directly. Advice, by contrast, is explicitly bidirectional and shared: every model contributes non-binding recommendations, and - this is stated with equal weight - **no model's advice, including Claude's own, carries authority by itself.** The user weighs it; Reality (tests, findings) settles it.

This grants no new authority over the trading system itself: every change Claude makes under this protocol still has to pass the same gates as any other change - the promotion pipeline (Chapter 16), the path-guard and confirm-gate (Chapter 21's Executor pattern, reused here), and explicit user approval for anything irreversible.

GATHER / ENHANCE / IMPLEMENT

Every work cycle follows a three-phase shape: **GATHER** (map affected files and constraints before touching code), **ENHANCE** (sharpen the prompt going to the next model - context, exact output format, an explicit "if unsure, write `# UNKNOWN:` - never invent" guard, role+constraint pairing), **IMPLEMENT** (a SpecWriter -> parallel DiffGen/TestGen -> Auditor chain, ending in commit/PR/changelog writers). Work moves through six status codes - `PENDING`, `IN_PROGRESS`, `BLOCKED`, `UNKNOWN`, `DONE`, `NEEDS REVISION` - tracked per cycle in a lightweight state object, with the *authoritative* record living in `multi_llm/turn_ledger.jsonl` and `HANDOFF.md`, not the convenience state object itself.

The infrastructure underneath

- **`context/*.md` - the "Portable Mind."** Generated, gitignored derived views regenerated by `python scripts/context/build_context.py`. Truth stays in the repo (`CLAUDE.md`, `docs/current-findings.md`, the queue); these files are never hand-edited.
- **`multi_llm/build_queue.jsonl`** - the single backlog feeding what each model works on next.
- **`multi_llm/turn_ledger.jsonl`** - every model turn, captured verbatim, indexed for rewind and digest tooling (`scripts/context/discussion.py`).
- **Two session logs**, distinguished by dimension, not chronology: `assistant_project.md` for codebase/engineering work (this is the log every chapter of this book, and every response in the session that produced it, is required to append to - see `CLAUDE.md` Section 6), and `llm_project_assistant.md` for the *operation* of the multi-LLM workflow itself (handoff cycles, cross-model coordination decisions).

Classification

Concept	Status
The 6-model role pipeline + handoff protocol	Production process (human-operated, not automated)
Code & Execution Authority rule (Claude-exclusive)	Production doctrine
`context/*.md` Portable Mind	Production tooling, generated/gitignored
`multi_llm/turn_ledger.jsonl`, `build_queue.jsonl`	Production, authoritative state

Authoritative sources

- `CLAUDE.md` Section 13 - the full pipeline, role cards, phase framework, status codes, state object shape, and the Code & Execution Authority rule, verbatim.
- `multi_llm/MULTI_LLM_PROTOCOL.md`, `multi_llm/README.md` - the canonical protocol spec.
- `multi_llm/roles/` - the five per-agent role files (`ROLE_CHATGPT.md`, `ROLE_CLAUDE.md`, `ROLE_DEEPSEEK.md`, `ROLE_GEMINI.md`, `ROLE_USER.md`).
- `src/multi_llm/` - the code layer: `context_pack.py`, `discussion.py`, `tokens.py`, `turn_ledger.py`.
- `ChatGpt workflow/OrchestartedPrompts/` - plain-text/markdown prompt-stage artifacts (the rest of that directory is largely `.docx` illustrative material, not an authoritative source).
- `tests/test_context_compiler.py`, `tests/test_handoff_state.py` - the enforcement floors.

Unresolved questions

None - `CLAUDE.md` Section 13 is an unusually complete, self-contained primary source for this entire layer.

Previous: Chapter 22 - The Control Plane | **Next:** Appendix A1 - Testing the System **Related:** Chapter 21 - The AI Automation Agent (a different, in-repo agent layer - don't conflate the two) **Memory:** none dedicated for this session.

Appendix

Appendix A1 - Testing the System

Status of this chapter: Written

Why this chapter exists

Nearly every earlier chapter cited a test file as part of a concept's "Authoritative sources" - this appendix is the one place that explains how the test suite as a whole is organized, so those citations make sense in context.

What you need to already know

Any Part II-VIII chapter - this is reference material for all of them, not a narrative continuation.

The idea

Running it

`docs/reference/testing.md` documents running the entire suite, filtering by domain (illustrative examples given for the Agent layer, Engine orchestration, Governance gates, the Execution Planner, Risk gates, and the live-executor subpackage), running a single test, and running with coverage.

Layout

The suite mirrors `src/`'s structure. Some domains live as dedicated subdirectories - `tests/analytics/`, `tests/cognitive/`, `tests/config_layer/`, `tests/data_ingestion/`, `tests/engines/`, `tests/events/`, `tests/exec_telemetry/`, `tests/execution/`, `tests/features/`, `tests/governance/`, `tests/inout/`, `tests/interpreters/`, `tests/journal/`, `tests/mt5_analytics/`, `tests/portfolio/`, `tests/regime/`, `tests/replay/`, `tests/research/`, `tests/runtime/` - while others (Agent, most of Engine orchestration, Governance gates, Planner/Risk) live as flat `test_*.py` files directly under `tests/`, not in a dedicated subdirectory. `tests/fixtures/`, `tests/golden/`, `tests/harness/`, `tests/helpers/`, `tests/manual/`, `tests/production_configs/` hold shared test infrastructure rather than test cases themselves.

Representative patterns

The reference doc documents four representative test-writing patterns (assertion-based, invariant- based, parametrized, and registry-exhaustiveness testing), the conventions the suite enforces beyond just passing (see Chapter 3's classification vocabulary - many of this book's "AUDITED" and "CLOSED" statuses trace back to a specific test file asserting the boundary they describe), and a guide for writing a new test.

A doc-drift this appendix surfaces

`docs/reference/testing.md`'s own header states a count - "116 files across 15 directories," recorded 2026-06-12. A direct directory count taken while researching this book found roughly **1,200+ files under**

`tests/` - an order of magnitude higher. This is almost certainly the reference doc's count simply not having been refreshed as the suite grew, not a discrepancy in the suite itself; flagged here per the same `DOC_DRIFT` discipline as Chapter 7's 38-vs-39-dim finding, and left unfixed by this book for the same reason - that's a separate, explicit doc-drift turn.

Classification

Concept	Status
The test suite itself	Production, actively maintained
`docs/reference/testing.md`'s file-count header	**Stale** - `DOC_DRIFT`, count last refreshed 2026-06-12

Authoritative sources

- `docs/reference/testing.md` - the full reference (running, config, layout, coverage expectations, patterns, conventions, CI/regression).
- `tests/conftest.py` - shared fixtures and pytest configuration hooks.
- `pyproject.toml` - the pytest configuration itself.

Unresolved questions

- The exact current file/directory count for `tests/` should be re-verified with a fresh count before `testing.md`'s header is corrected - this appendix's "~1,200+" figure is itself an approximate count taken during this book's research, not a precise audit.

Previous: Chapter 23 - Multi-LLM Coordination | **Next:** Appendix A2 - Unresolved Questions **Related:** Chapter 03 - How This Book Fits **Memory:** none dedicated.

Appendix A2 - Unresolved Questions (Rollup)

Status of this chapter: Written

Why this chapter exists

Per this book's own charter (see the task brief this book was built from): "if understanding is impossible, do not invent." Every chapter that hit a genuine gap in evidence recorded it locally rather than guessing. This appendix collects all of those in one place, so a future session extending the book - or fixing a real issue this research surfaced - has a single punch list instead of needing to re-read every chapter to find the open threads.

What you need to already know

This is a reference appendix, not a narrative chapter - each item below links back to the chapter that raised it, where the full context lives.

The rollup

Book-scope gaps (things the book hasn't covered yet, not defects in the repo)

- **22 `src/` subpackages have no dedicated chapter yet** - including several explicitly `DORMANT` (Cognitive bus, Strategies S01-S10, Scanner, Feedback, LLM research, Monitoring, Journal, Data ingestion, UAT) and several not yet classified either way (`expansion/`, `replay/`, `portfolio/`, `regime/`, `retrieval/`, `msip/`, `analytics/`, `events/`, `validation_access/`, etc.). See Chapter 4.
- **MSIP (Market State Interpretation Platform)** is a large enough program - its own design contract, implementation plan, and pre-registration series - to arguably deserve its own chapter. See Chapter 18.
- **Program 9** appears in recent commit history (M1-M5 ladder, OCO straddle kernel, Stage-1/Stage-2 drivers) but wasn't present in the Funding Ledger structure this book's research verified. Its hypothesis, scope, and status are undocumented in this book. See Chapter 19. **This is the single most likely candidate for the next chapter to write**, since it's active, recent work with no book coverage at all.
- **Program 7 (Open Interest)** - named as out-of-scope for Program 6 but its own status (planned, started, or not) wasn't independently verified. See Chapter 19.
- Parts VI-VIII (chapters 16-23) are, by this pass's own explicit scoping decision, orientation-level rather than full depth - deepening any of them is expected future book work, not a defect.

Real repository issues this research surfaced (not book gaps - code/doc issues)

- **`docs/reference/schemas.md` Section 4 still documents the 38-dim v3.0 feature tuple; the live code is 39-dim v4.0.** A textbook `DOC_DRIFT` fix, low-risk, not done as part of this book-writing pass by design. See Chapter 7.

- **`docs/reference/testing.md`'s file-count header ("116 files across 15 directories," dated 2026-06-12) is stale** - a direct count during this research found roughly 1,200+ files under `tests/`. See Appendix A1.
- **The ATR unit mismatch in `compute_crt_levels`** (SL/TP geometry computed against a close-relative `atr` value as if it were an absolute price) - a real, previously-identified defect, currently **unfixed**, awaiting an explicit user remediation decision. See Chapter 13.
- **F-057's `CRTConfig` programmatic-path split-brain** remains **OPEN** - production JSON config is bypassed on any `BacktestRunner(cfg)` call with `crt_config=None`. See Chapter 13.
- **The relationship between `src/inout/` (current, 11 files) and `archive/inout_legacy/` (the rail `goal.md` describes as "currently archived") is unclear.** Needs a direct read of both to resolve. See Chapter 15.

Verification gaps (things assumed true from documentation, not independently re-checked)

- Whether Wyckoff, Market Profile, or Order Flow interpreters have any code beyond scaffolding, and how far along the Interpreter Contract pipeline they are. See Chapter 9.
- Whether the Research Measurement Contract has any sealed `MC-*` instances beyond the last verified count (zero, as of the evidence this book drew on). See Chapter 20.
- Whether every tool documented in `docs/reference/agent-reference.md` is currently reachable end-to-end versus still scaffolded. See Chapter 21.
- `docs/reference/architecture.md`'s full body (design patterns, external integrations, entry points) was header-verified but not read in full. See Chapter 4.

Suggested follow-up (proposed, not yet actioned)

Per this book's own build plan, one small additional step was flagged but deliberately not done automatically: **adding a pointer row for `docs/book/README.md` to `CLAUDE.md` Section 2's Companion Documentation table**, so future sessions discover the book without being told about it explicitly. This is a one-line, low-risk addition - but `CLAUDE.md` is itself governed content, so it's proposed here rather than done silently.

Previous: Appendix A1 - Testing the System | **Next:** - (last chapter; return to the Table of Contents) **Related:** every chapter linked above. **Memory:** none dedicated - this appendix is the book's own open-loop tracker.

Front Matter **3**
The Tradelatest Book **4**
Part I - Foundations **9**
Chapter 01 - Why Tradelatest Exists **10**
Chapter 02 - The Invariants and the Happy Flow **13**
Chapter 03 - How This Book Fits the Repository's Knowledge **17**
Chapter 04 - Architecture at a Glance **21**
Part II - Market Understanding **25**